JAVASCRIPT Interview Jargon Handbook

1,200+ questions with concise answers, runtime mental models, and senior-level trade-offs

Built as a revision reference for experienced frontend engineers

How to use this handbook

For senior interviews, do not stop at the definition. Use each question as a prompt to explain what happens at runtime, why the feature exists, the important edge cases, and the trade-off you would make in production. The answers are intentionally compact so you can expand them aloud.

Contents

1. JavaScript Fundamentals
2. Scope, Execution Context and Call Stack
3. Hoisting, var/let/const and TDZ
4. Closures
5. this, call, apply and bind
6. Prototypes, new and Inheritance
7. Functions and Functional Concepts
8. Objects and Property Descriptors
9. Arrays and Collections
10. Promise Internals
11. async/await
12. Event Loop and Scheduling
13. Fetch, Network and Cancellation
14. DOM and Browser Events
15. Browser Rendering
16. JavaScript Engines and JIT
17. Garbage Collection and Memory
18. Modules and Loading
19. Node.js Runtime
20. Errors and Resilience
21. Serialization and Data Transfer
22. Symbols, Iterators and Generators
23. Proxy and Reflect
24. Debounce, Throttle and Async Patterns
25. Immutability and Equality
26. Classes and Private Fields
27. Browser Storage
28. Web Security
29. Workers and Browser Concurrency
30. Browser Observers and APIs
31. Networking and Caching
32. Web Performance Metrics
33. SSR, SSG, ISR and Hydration
34. React Server Components and Rendering
35. React Internals and Rendering
36. React State, Context and Hooks
37. Architecture and Design
38. Testing
39. Bundling and Tooling
40. Senior Performance and Memory
41. Code Output and Scheduling Drills
42. High-Frequency Comparison Questions
43. Senior JavaScript Why/Trade-off Questions
44. Senior Async and Concurrency Scenarios
45. Senior Browser Performance Scenarios
46. Senior React/Frontend Scenarios
47. Senior Security Scenarios
48. Senior System Design Vocabulary
49. Metaprogramming and Language Semantics
50. Date, Number and Internationalization
51. Regular Expressions
52. Iteration Patterns
53. Advanced Promise and Concurrency Questions
54. Modern JavaScript Architecture
55. Web Storage, Offline and PWA
56. Debugging and DevTools
57. Frontend Networking and Caching Scenarios

1. JavaScript Fundamentals

50 interview prompts

1. What is javascript?

Answer

JavaScript is a high-level, dynamically typed language standardized by ECMAScript. Browsers and runtimes such as Node.js execute it through an engine plus host-provided APIs.

2. How does javascript work?

Answer

JavaScript is a high-level, dynamically typed language standardized by ECMAScript. Browsers and runtimes such as Node.js execute it through an engine plus host-provided APIs. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

3. What are common interview traps with javascript?

Answer

JavaScript is a high-level, dynamically typed language standardized by ECMAScript. Browsers and runtimes such as Node.js execute it through an engine plus host-provided APIs. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

4. What is dynamic typing?

Answer

Dynamic typing means a variable does not have a fixed declared type; the value currently stored determines the runtime type, and that type can change over time.

5. Why does dynamic typing matter in production code?

Answer

Dynamic typing means a variable does not have a fixed declared type; the value currently stored determines the runtime type, and that type can change over time. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

6. What is type coercion?

Answer

Type coercion is converting a value from one type to another. JavaScript can do this implicitly for operators such as ==, +, and relational comparisons, or explicitly via Number, String, Boolean, etc.

7. How does type coercion work?

Answer

Type coercion is converting a value from one type to another. JavaScript can do this implicitly for operators such as ==, +, and relational comparisons, or explicitly via Number, String, Boolean, etc. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

8. What is implicit coercion?

Answer

Implicit coercion is type conversion performed by the language during an operation rather than by an explicit conversion call.

9. Why does implicit coercion matter in production code?

Answer

Implicit coercion is type conversion performed by the language during an operation rather than by an explicit conversion call. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

10. What are common interview traps with implicit coercion?

Answer

Implicit coercion is type conversion performed by the language during an operation rather than by an explicit conversion call. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

11. What is explicit coercion?

Answer

Explicit coercion is a deliberate conversion such as Number("42"), String(42), Boolean(value), or BigInt("42").

12. How does explicit coercion work?

Answer

Explicit coercion is a deliberate conversion such as Number("42"), String(42), Boolean(value), or BigInt("42"). Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

13. What is primitive?

Answer

JavaScript primitive values are immutable values: string, number, bigint, boolean, undefined, symbol, and null. They are compared by value rather than object identity.

14. Why does primitive matter in production code?

Answer

JavaScript primitive values are immutable values: string, number, bigint, boolean, undefined, symbol, and null. They are compared by value rather than object identity. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

15. What is reference value?

Answer

Objects, arrays, and functions are reference values in common interview terminology: a variable holds a reference to an object, while the object itself has identity and mutable state.

16. How does reference value work?

Answer

Objects, arrays, and functions are reference values in common interview terminology: a variable holds a reference to an object, while the object itself has identity and mutable state. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

17. What are common interview traps with reference value?

Answer

Objects, arrays, and functions are reference values in common interview terminology: a variable holds a reference to an object, while the object itself has identity and mutable state. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

18. What is pass by value?

Answer

JavaScript is pass-by-value. For objects, the value being passed is a reference to the object, so mutating the object is visible to the caller, but reassigning the parameter is not.

19. Why does pass by value matter in production code?

Answer

JavaScript is pass-by-value. For objects, the value being passed is a reference to the object, so mutating the object is visible to the caller, but reassigning the parameter is not. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

20. What is null?

Answer

null is an intentional absence of an object/value. It is a primitive value, although typeof null historically returns "object".

21. How does null work?

Answer

null is an intentional absence of an object/value. It is a primitive value, although typeof null historically returns "object". Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

22. What is undefined?

Answer

undefined means a value is absent or has not been assigned. It is also the default result of many operations that do not produce a value.

23. Why does undefined matter in production code?

Answer

undefined means a value is absent or has not been assigned. It is also the default result of many operations that do not produce a value. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

24. What are common interview traps with undefined?

Answer

undefined means a value is absent or has not been assigned. It is also the default result of many operations that do not produce a value. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

25. What is nan?

Answer

NaN means "Not-a-Number", but its ECMAScript type is Number. It is the result of invalid numeric operations and is the only JavaScript value that is not equal to itself.

26. How does nan work?

Answer

NaN means "Not-a-Number", but its ECMAScript type is Number. It is the result of invalid numeric operations and is the only JavaScript value that is not equal to itself. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

27. What is truthy?

Answer

Truthy values become true in Boolean contexts. Everything is truthy except false, 0, -0, 0n, NaN, null, undefined, and the empty string.

28. Why does truthy matter in production code?

Answer

Truthy values become true in Boolean contexts. Everything is truthy except false, 0, -0, 0n, NaN, null, undefined, and the empty string. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

29. What is falsy?

Answer

Falsy values are false, 0, -0, 0n, NaN, null, undefined, and "". Objects and arrays are truthy, including empty ones.

30. How does falsy work?

Answer

Falsy values are false, 0, -0, 0n, NaN, null, undefined, and "". Objects and arrays are truthy, including empty ones. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

31. What are common interview traps with falsy?

Answer

Falsy values are false, 0, -0, 0n, NaN, null, undefined, and "". Objects and arrays are truthy, including empty ones. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

32. What is short-circuit evaluation?

Answer

Logical operators evaluate left-to-right and stop when the result is already determined. This lets &&, ||, and ?? act as control-flow-like expressions.

33. Why does short-circuit evaluation matter in production code?

Answer

Logical operators evaluate left-to-right and stop when the result is already determined. This lets &&, ||, and ?? act as control-flow-like expressions. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

34. What is nullish coalescing?

Answer

?? returns the right side only when the left side is null or undefined. Unlike ||, it does not treat 0, false, or "" as missing.

35. How does nullish coalescing work?

Answer

?? returns the right side only when the left side is null or undefined. Unlike ||, it does not treat 0, false, or "" as missing. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

36. What is optional chaining?

Answer

?. safely accesses a property, call, or index only when the value on its left is not nullish; otherwise the expression evaluates to undefined.

37. Why does optional chaining matter in production code?

Answer

?. safely accesses a property, call, or index only when the value on its left is not nullish; otherwise the expression evaluates to undefined. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

38. What are common interview traps with optional chaining?

Answer

?. safely accesses a property, call, or index only when the value on its left is not nullish; otherwise the expression evaluates to undefined. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

39. What is boxing?

Answer

Boxing is the temporary wrapping of a primitive in an object-like wrapper so property access or methods can work, such as "hi".toUpperCase().

40. How does boxing work?

Answer

Boxing is the temporary wrapping of a primitive in an object-like wrapper so property access or methods can work, such as "hi".toUpperCase(). Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

41. Why is typeof null "object"?

Answer

An object is a collection of properties identified by keys. Property values can be data, accessors, or functions, and the object can delegate through a prototype. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

42. Why is typeof NaN "number"?

Answer

NaN means "Not-a-Number", but its ECMAScript type is Number. It is the result of invalid numeric operations and is the only JavaScript value that is not equal to itself. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

43. Why is 0.1 + 0.2 not exactly 0.3?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

44. Why are objects and arrays compared by reference?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

45. What is the difference between == and ===?

Answer

== and === solve related problems but differ in semantics and trade-offs. The distinction depends on the API semantics and execution model. Choose based on the exact behavior, lifecycle, and performance constraints required.

46. What is Object.is()?

Answer

Object.is uses SameValue semantics: it treats NaN as equal to NaN and distinguishes +0 from -0. For most ordinary object comparisons, it still compares object identity.

47. How does + behave with strings and numbers?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

48. What is BigInt and when should you use it?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

49. What is -0?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

50. How do you reliably detect NaN?

Answer

NaN means "Not-a-Number", but its ECMAScript type is Number. It is the result of invalid numeric operations and is the only JavaScript value that is not equal to itself. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

2. Scope, Execution Context and Call Stack

35 interview prompts

51. What is scope?

Answer

Scope determines where an identifier can be resolved. JavaScript has global, module, function, and block scopes, with lexical scope based on source-code nesting.

52. How does scope work?

Answer

Scope determines where an identifier can be resolved. JavaScript has global, module, function, and block scopes, with lexical scope based on source-code nesting. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

53. What are common interview traps with scope?

Answer

Scope determines where an identifier can be resolved. JavaScript has global, module, function, and block scopes, with lexical scope based on source-code nesting. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

54. What is lexical scope?

Answer

Lexical scope means identifier visibility is determined by where code is written, not by where a function is called.

55. Why does lexical scope matter in production code?

Answer

Lexical scope means identifier visibility is determined by where code is written, not by where a function is called. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

56. What is function scope?

Answer

A var declaration is function-scoped: its binding belongs to the nearest function or the global scope rather than the nearest block.

57. How does function scope work?

Answer

A var declaration is function-scoped: its binding belongs to the nearest function or the global scope rather than the nearest block. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

58. What is block scope?

Answer

let, const, and class declarations are block-scoped, so their bindings exist only inside the containing block.

59. Why does block scope matter in production code?

Answer

let, const, and class declarations are block-scoped, so their bindings exist only inside the containing block. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

60. What are common interview traps with block scope?

Answer

let, const, and class declarations are block-scoped, so their bindings exist only inside the containing block. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

61. What is module scope?

Answer

Top-level declarations in an ES module belong to that module and are not automatically added to the global object.

62. How does module scope work?

Answer

Top-level declarations in an ES module belong to that module and are not automatically added to the global object. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

63. What is scope chain?

Answer

The scope chain is the sequence of lexical environments searched from the current scope outward when resolving an identifier.

64. Why does scope chain matter in production code?

Answer

The scope chain is the sequence of lexical environments searched from the current scope outward when resolving an identifier. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

65. What is execution context?

Answer

An execution context is the specification-level state used while evaluating code. It tracks things such as lexical environments and the current execution state. The call stack holds active execution contexts.

66. How does execution context work?

Answer

An execution context is the specification-level state used while evaluating code. It tracks things such as lexical environments and the current execution state. The call stack holds active execution contexts. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

67. What are common interview traps with execution context?

Answer

An execution context is the specification-level state used while evaluating code. It tracks things such as lexical environments and the current execution state. The call stack holds active execution contexts. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

68. What is call stack?

Answer

The call stack tracks currently executing function calls. JavaScript execution is run-to-completion on a given thread: one stack frame runs until it returns or throws.

69. Why does call stack matter in production code?

Answer

The call stack tracks currently executing function calls. JavaScript execution is run-to-completion on a given thread: one stack frame runs until it returns or throws. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

70. What is lexical environment?

Answer

A lexical environment pairs an environment record containing bindings with a reference to an outer lexical environment. It is the foundation for lexical scoping and closures.

71. How does lexical environment work?

Answer

A lexical environment pairs an environment record containing bindings with a reference to an outer lexical environment. It is the foundation for lexical scoping and closures. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

72. What is environment record?

Answer

An environment record stores identifier bindings for an execution context. Different record types model declarative bindings, object-backed bindings, and module bindings.

73. Why does environment record matter in production code?

Answer

An environment record stores identifier bindings for an execution context. Different record types model declarative bindings, object-backed bindings, and module bindings. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

74. What are common interview traps with environment record?

Answer

An environment record stores identifier bindings for an execution context. Different record types model declarative bindings, object-backed bindings, and module bindings. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

75. What is outer lexical environment?

Answer

Each lexical environment can reference an outer environment. Variable resolution walks this chain until a matching binding is found or a ReferenceError is produced.

76. How does outer lexical environment work?

Answer

Each lexical environment can reference an outer environment. Variable resolution walks this chain until a matching binding is found or a ReferenceError is produced. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

77. What happens when a function is called?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

78. How does variable lookup work?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

79. What happens when a variable is not found?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

80. How is an execution context related to the call stack?

Answer

An execution context is the specification-level state used while evaluating code. It tracks things such as lexical environments and the current execution state. The call stack holds active execution contexts. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

81. What is the global execution context?

Answer

An execution context is the specification-level state used while evaluating code. It tracks things such as lexical environments and the current execution state. The call stack holds active execution contexts.

82. What is a function execution context?

Answer

An execution context is the specification-level state used while evaluating code. It tracks things such as lexical environments and the current execution state. The call stack holds active execution contexts.

83. Why is lexical scope important?

Answer

Lexical scope means identifier visibility is determined by where code is written, not by where a function is called. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

84. Is scope determined at definition time or call time?

Answer

Scope determines where an identifier can be resolved. JavaScript has global, module, function, and block scopes, with lexical scope based on source-code nesting.

85. How do nested functions find outer variables?

Answer

find returns the first element satisfying a predicate, or undefined if none does. It stops once a match is found. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

3. Hoisting, var/let/const and TDZ

27 interview prompts

86. What is hoisting?

Answer

Hoisting is an interview shorthand for the fact that declarations are processed before the corresponding execution reaches their source position. It is not literal source-code movement.

87. How does hoisting work?

Answer

Hoisting is an interview shorthand for the fact that declarations are processed before the corresponding execution reaches their source position. It is not literal source-code movement. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

88. What are common interview traps with hoisting?

Answer

Hoisting is an interview shorthand for the fact that declarations are processed before the corresponding execution reaches their source position. It is not literal source-code movement. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

89. What is temporal dead zone?

Answer

The TDZ is the period from entering a scope until a let, const, or class binding is initialized. Accessing the binding in that interval throws a ReferenceError.

90. Why does temporal dead zone matter in production code?

Answer

The TDZ is the period from entering a scope until a let, const, or class binding is initialized. Accessing the binding in that interval throws a ReferenceError. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

91. What is declaration?

Answer

A declaration introduces a binding, such as let x or function f(). Initialization assigns its initial value; those are distinct phases for let/const.

92. How does declaration work?

Answer

A declaration introduces a binding, such as let x or function f(). Initialization assigns its initial value; those are distinct phases for let/const. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

93. What is initialization?

Answer

Initialization is the step where a newly created binding receives its initial value. const must be initialized at declaration; let is initialized when execution reaches the declaration.

94. Why does initialization matter in production code?

Answer

Initialization is the step where a newly created binding receives its initial value. const must be initialized at declaration; let is initialized when execution reaches the declaration. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

95. What are common interview traps with initialization?

Answer

Initialization is the step where a newly created binding receives its initial value. const must be initialized at declaration; let is initialized when execution reaches the declaration. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

96. What is var?

Answer

var is function-scoped and its binding is initialized to undefined during environment setup; redeclaration is allowed in the same scope.

97. How does var work?

Answer

var is function-scoped and its binding is initialized to undefined during environment setup; redeclaration is allowed in the same scope. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

98. What is let?

Answer

let is block-scoped. Its binding exists before the declaration is executed but remains in the temporal dead zone until initialization.

99. Why does let matter in production code?

Answer

let is block-scoped. Its binding exists before the declaration is executed but remains in the temporal dead zone until initialization. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

100. What is const?

Answer

const is block-scoped and requires initialization at declaration. The binding itself cannot be reassigned, but referenced objects can still be mutated unless they are separately frozen.

101. How does const work?

Answer

const is block-scoped and requires initialization at declaration. The binding itself cannot be reassigned, but referenced objects can still be mutated unless they are separately frozen. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

102. What are common interview traps with const?

Answer

const is block-scoped and requires initialization at declaration. The binding itself cannot be reassigned, but referenced objects can still be mutated unless they are separately frozen. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

103. Are let and const hoisted?

Answer

const is block-scoped and requires initialization at declaration. The binding itself cannot be reassigned, but referenced objects can still be mutated unless they are separately frozen.

104. Why can accessing let before declaration throw?

Answer

A declaration introduces a binding, such as let x or function f(). Initialization assigns its initial value; those are distinct phases for let/const. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

105. Why does var return undefined before its declaration?

Answer

A declaration introduces a binding, such as let x or function f(). Initialization assigns its initial value; those are distinct phases for let/const. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

106. What happens when var is declared inside a block?

Answer

var is function-scoped and its binding is initialized to undefined during environment setup; redeclaration is allowed in the same scope.

107. Can const objects be mutated?

Answer

const is block-scoped and requires initialization at declaration. The binding itself cannot be reassigned, but referenced objects can still be mutated unless they are separately frozen.

108. What is the difference between declaration and initialization?

Answer

declaration and initialization solve related problems but differ in semantics and trade-offs. A declaration introduces a binding, such as let x or function f(). Initialization assigns its initial value; those are distinct phases for let/const. Initialization is the step where a newly created binding receives its initial value. const must be initialized at declaration; let is initialized when execution reaches the declaration.

109. How are function declarations hoisted?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

110. How are function expressions hoisted?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

111. Can you redeclare let in the same scope?

Answer

Scope determines where an identifier can be resolved. JavaScript has global, module, function, and block scopes, with lexical scope based on source-code nesting.

112. Why prefer const over var in modern code?

Answer

const is block-scoped and requires initialization at declaration. The binding itself cannot be reassigned, but referenced objects can still be mutated unless they are separately frozen. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

4. Closures

17 interview prompts

113. What is closure?

Answer

A closure is a function together with access to the lexical environment where it was created. It lets the function continue using outer variables after the outer function has returned.

114. How does closure work?

Answer

A closure is a function together with access to the lexical environment where it was created. It lets the function continue using outer variables after the outer function has returned. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

115. What are common interview traps with closure?

Answer

A closure is a function together with access to the lexical environment where it was created. It lets the function continue using outer variables after the outer function has returned. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

116. How does scope chain work?

Answer

The scope chain is the sequence of lexical environments searched from the current scope outward when resolving an identifier. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

117. What is memory leak?

Answer

A JavaScript memory leak occurs when objects remain reachable even though the application no longer needs them, so garbage collection cannot reclaim them.

118. Why does memory leak matter in production code?

Answer

A JavaScript memory leak occurs when objects remain reachable even though the application no longer needs them, so garbage collection cannot reclaim them. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

119. What are common interview traps with memory leak?

Answer

A JavaScript memory leak occurs when objects remain reachable even though the application no longer needs them, so garbage collection cannot reclaim them. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

120. What exactly does a closure capture?

Answer

A closure is a function together with access to the lexical environment where it was created. It lets the function continue using outer variables after the outer function has returned.

121. Does a closure copy a variable or keep access to the binding?

Answer

A closure is a function together with access to the lexical environment where it was created. It lets the function continue using outer variables after the outer function has returned.

122. Can closures cause memory leaks?

Answer

A JavaScript memory leak occurs when objects remain reachable even though the application no longer needs them, so garbage collection cannot reclaim them.

123. Why does the classic var loop example print the final value?

Answer

var is function-scoped and its binding is initialized to undefined during environment setup; redeclaration is allowed in the same scope. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

124. How does let fix the loop closure problem?

Answer

A closure is a function together with access to the lexical environment where it was created. It lets the function continue using outer variables after the outer function has returned. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

125. Implement a counter with closure.

Answer

A closure is a function together with access to the lexical environment where it was created. It lets the function continue using outer variables after the outer function has returned.

126. Implement once() using closure.

Answer

A closure is a function together with access to the lexical environment where it was created. It lets the function continue using outer variables after the outer function has returned.

127. Implement memoize() using closure.

Answer

A closure is a function together with access to the lexical environment where it was created. It lets the function continue using outer variables after the outer function has returned.

128. Why are closures useful for private state?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

129. How would you debug a closure-related memory leak?

Answer

A JavaScript memory leak occurs when objects remain reachable even though the application no longer needs them, so garbage collection cannot reclaim them. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

5. this, call, apply and bind

24 interview prompts

130. What is this?

Answer

this is a dynamically determined reference used by normal functions. For calls made as object methods, constructors, or with call/apply/bind, the call form determines this; arrow functions capture this lexically.

131. How does this work?

Answer

this is a dynamically determined reference used by normal functions. For calls made as object methods, constructors, or with call/apply/bind, the call form determines this; arrow functions capture this lexically. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

132. What are common interview traps with this?

Answer

this is a dynamically determined reference used by normal functions. For calls made as object methods, constructors, or with call/apply/bind, the call form determines this; arrow functions capture this lexically. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

133. What is arrow function?

Answer

Arrow functions do not create their own this, arguments, or new.target. They capture these from the surrounding lexical context and cannot be used with new.

134. Why does arrow function matter in production code?

Answer

Arrow functions do not create their own this, arguments, or new.target. They capture these from the surrounding lexical context and cannot be used with new. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

135. What is call apply bind?

Answer

call invokes a function with an explicit this and separate arguments; apply does the same with an argument array-like value; bind returns a new function with this and optionally leading arguments fixed.

136. How does call apply bind work?

Answer

call invokes a function with an explicit this and separate arguments; apply does the same with an argument array-like value; bind returns a new function with this and optionally leading arguments fixed. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

137. What is call?

Answer

Function.prototype.call invokes a function immediately with an explicitly supplied this value and individual arguments.

138. Why does call matter in production code?

Answer

Function.prototype.call invokes a function immediately with an explicitly supplied this value and individual arguments. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

139. What are common interview traps with call?

Answer

Function.prototype.call invokes a function immediately with an explicitly supplied this value and individual arguments. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

140. What is apply?

Answer

Function.prototype.apply invokes a function immediately with an explicitly supplied this value and arguments provided as an array-like object.

141. How does apply work?

Answer

Function.prototype.apply invokes a function immediately with an explicitly supplied this value and arguments provided as an array-like object. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

142. What is bind?

Answer

Function.prototype.bind creates a new function whose this and optionally leading arguments are fixed. Calling bind does not execute the function.

143. Why does bind matter in production code?

Answer

Function.prototype.bind creates a new function whose this and optionally leading arguments are fixed. Calling bind does not execute the function. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

144. What determines this for a regular function call?

Answer

this is a dynamically determined reference used by normal functions. For calls made as object methods, constructors, or with call/apply/bind, the call form determines this; arrow functions capture this lexically.

145. What is the default binding of this?

Answer

this is a dynamically determined reference used by normal functions. For calls made as object methods, constructors, or with call/apply/bind, the call form determines this; arrow functions capture this lexically.

146. What happens to this in strict mode?

Answer

Strict mode changes several JavaScript semantics, including disallowing accidental globals and making some invalid operations throw. ES modules are always strict.

147. Why does a detached method often lose this?

Answer

this is a dynamically determined reference used by normal functions. For calls made as object methods, constructors, or with call/apply/bind, the call form determines this; arrow functions capture this lexically. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

148. Why do arrow functions not have their own this?

Answer

Arrow functions do not create their own this, arguments, or new.target. They capture these from the surrounding lexical context and cannot be used with new. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

149. Can bind permanently change this?

Answer

this is a dynamically determined reference used by normal functions. For calls made as object methods, constructors, or with call/apply/bind, the call form determines this; arrow functions capture this lexically.

150. What happens when a bound function is called with call()?

Answer

Function.prototype.call invokes a function immediately with an explicitly supplied this value and individual arguments.

151. Can arrow functions be constructors?

Answer

Arrow functions do not create their own this, arguments, or new.target. They capture these from the surrounding lexical context and cannot be used with new.

152. How does this behave in a DOM event handler?

Answer

this is a dynamically determined reference used by normal functions. For calls made as object methods, constructors, or with call/apply/bind, the call form determines this; arrow functions capture this lexically. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

153. Why is this not the same as lexical scope?

Answer

Lexical scope means identifier visibility is determined by where code is written, not by where a function is called. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

6. Prototypes, new and Inheritance

19 interview prompts

154. What is prototype?

Answer

A function object typically has a prototype property used as the prototype assigned to instances created with new. Methods defined on a class are also stored on a prototype object.

155. How does prototype work?

Answer

A function object typically has a prototype property used as the prototype assigned to instances created with new. Methods defined on a class are also stored on a prototype object. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

156. What are common interview traps with prototype?

Answer

A function object typically has a prototype property used as the prototype assigned to instances created with new. Methods defined on a class are also stored on a prototype object. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

157. What is prototype chain?

Answer

Property lookup checks an object and then recursively follows its [[Prototype]] chain until a property is found or null is reached.

158. Why does prototype chain matter in production code?

Answer

Property lookup checks an object and then recursively follows its [[Prototype]] chain until a property is found or null is reached. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

159. What is __proto__?

Answer

__proto__ is a legacy accessor for an object's [[Prototype]]. Object.getPrototypeOf and Object.setPrototypeOf are the standard APIs for inspecting or changing it.

160. How does __proto__ work?

Answer

__proto__ is a legacy accessor for an object's [[Prototype]]. Object.getPrototypeOf and Object.setPrototypeOf are the standard APIs for inspecting or changing it. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

161. What is [[prototype]]?

Answer

[[Prototype]] is an internal slot linking an object to another object from which it can inherit properties.

162. Why does [[prototype]] matter in production code?

Answer

[[Prototype]] is an internal slot linking an object to another object from which it can inherit properties. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

163. What are common interview traps with [[prototype]]?

Answer

[[Prototype]] is an internal slot linking an object to another object from which it can inherit properties. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

164. What is object.create?

Answer

Object.create(proto) creates a new object whose [[Prototype]] is proto, optionally with property descriptors.

165. How does object.create work?

Answer

Object.create(proto) creates a new object whose [[Prototype]] is proto, optionally with property descriptors. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

166. What is new?

Answer

new creates an instance, links it to the constructor prototype, calls the constructor with the new object as this, and returns the object unless the constructor returns another non-primitive object.

167. Why does new matter in production code?

Answer

new creates an instance, links it to the constructor prototype, calls the constructor with the new object as this, and returns the object unless the constructor returns another non-primitive object. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

168. What is prototypal inheritance?

Answer

Prototypal inheritance means objects delegate property lookup to another object through [[Prototype]]. JavaScript classes provide syntax over this prototype-based mechanism.

169. How does prototypal inheritance work?

Answer

Prototypal inheritance means objects delegate property lookup to another object through [[Prototype]]. JavaScript classes provide syntax over this prototype-based mechanism. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

170. What are common interview traps with prototypal inheritance?

Answer

Prototypal inheritance means objects delegate property lookup to another object through [[Prototype]]. JavaScript classes provide syntax over this prototype-based mechanism. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

171. What is prototype pollution?

Answer

Prototype pollution is an attack or bug where attacker-controlled input mutates shared prototype objects, causing unexpected inherited properties or behavior across unrelated objects.

172. Why does prototype pollution matter in production code?

Answer

Prototype pollution is an attack or bug where attacker-controlled input mutates shared prototype objects, causing unexpected inherited properties or behavior across unrelated objects. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

7. Functions and Functional Concepts

26 interview prompts

173. What is first-class function?

Answer

Functions are first-class values because they can be stored in variables, passed as arguments, returned from functions, and placed in data structures.

174. How does first-class function work?

Answer

Functions are first-class values because they can be stored in variables, passed as arguments, returned from functions, and placed in data structures. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

175. What are common interview traps with first-class function?

Answer

Functions are first-class values because they can be stored in variables, passed as arguments, returned from functions, and placed in data structures. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

176. What is higher-order function?

Answer

A higher-order function takes one or more functions as arguments or returns a function. map, filter, and reduce are common examples.

177. Why does higher-order function matter in production code?

Answer

A higher-order function takes one or more functions as arguments or returns a function. map, filter, and reduce are common examples. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

178. What is callback?

Answer

A callback is a function passed to another function or API so it can be invoked later or during a particular operation.

179. How does callback work?

Answer

A callback is a function passed to another function or API so it can be invoked later or during a particular operation. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

180. What is pure function?

Answer

A pure function returns the same output for the same inputs and has no observable side effects outside its return value.

181. Why does pure function matter in production code?

Answer

A pure function returns the same output for the same inputs and has no observable side effects outside its return value. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

182. What are common interview traps with pure function?

Answer

A pure function returns the same output for the same inputs and has no observable side effects outside its return value. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

183. What is impure function?

Answer

An impure function can depend on changing external state or cause observable side effects such as mutation, I/O, logging, or DOM updates.

184. How does impure function work?

Answer

An impure function can depend on changing external state or cause observable side effects such as mutation, I/O, logging, or DOM updates. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

185. What is currying?

Answer

Currying transforms a multi-argument function into a chain of single-argument functions, such as f(a,b,c) becoming f(a)(b)(c).

186. Why does currying matter in production code?

Answer

Currying transforms a multi-argument function into a chain of single-argument functions, such as f(a,b,c) becoming f(a)(b)(c). Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

187. What is partial application?

Answer

Partial application pre-fills some arguments and returns a new function waiting for the remaining arguments. It does not require one argument per function call.

188. How does partial application work?

Answer

Partial application pre-fills some arguments and returns a new function waiting for the remaining arguments. It does not require one argument per function call. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

189. What are common interview traps with partial application?

Answer

Partial application pre-fills some arguments and returns a new function waiting for the remaining arguments. It does not require one argument per function call. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

190. What is function composition?

Answer

Function composition combines functions so the output of one becomes the input of another, enabling small reusable transformations.

191. Why does function composition matter in production code?

Answer

Function composition combines functions so the output of one becomes the input of another, enabling small reusable transformations. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

192. What is iife?

Answer

An IIFE is an immediately invoked function expression. Before ES modules, it was commonly used to create a private scope and avoid global-name collisions.

193. How does iife work?

Answer

An IIFE is an immediately invoked function expression. Before ES modules, it was commonly used to create a private scope and avoid global-name collisions. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

194. What is recursion?

Answer

Recursion is when a function directly or indirectly calls itself, usually with a base case that stops further calls.

195. Why does recursion matter in production code?

Answer

Recursion is when a function directly or indirectly calls itself, usually with a base case that stops further calls. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

196. What are common interview traps with recursion?

Answer

Recursion is when a function directly or indirectly calls itself, usually with a base case that stops further calls. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

197. What is tail recursion?

Answer

Tail recursion occurs when a recursive call is the final operation of a function. Some languages optimize it, but JavaScript engines should not be assumed to provide general tail-call optimization in practice.

198. How does tail recursion work?

Answer

Tail recursion occurs when a recursive call is the final operation of a function. Some languages optimize it, but JavaScript engines should not be assumed to provide general tail-call optimization in practice. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

8. Objects and Property Descriptors

26 interview prompts

199. What is object?

Answer

An object is a collection of properties identified by keys. Property values can be data, accessors, or functions, and the object can delegate through a prototype.

200. How does object work?

Answer

An object is a collection of properties identified by keys. Property values can be data, accessors, or functions, and the object can delegate through a prototype. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

201. What are common interview traps with object?

Answer

An object is a collection of properties identified by keys. Property values can be data, accessors, or functions, and the object can delegate through a prototype. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

202. What is property descriptor?

Answer

A property descriptor records attributes such as value, writable, enumerable, configurable, get, and set. Object.defineProperty can define these explicitly.

203. Why does property descriptor matter in production code?

Answer

A property descriptor records attributes such as value, writable, enumerable, configurable, get, and set. Object.defineProperty can define these explicitly. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

204. What is enumerable?

Answer

Enumerable determines whether a property is included by enumeration mechanisms such as Object.keys and for...in, subject to their own rules.

205. How does enumerable work?

Answer

Enumerable determines whether a property is included by enumeration mechanisms such as Object.keys and for...in, subject to their own rules. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

206. What is writable?

Answer

Writable determines whether a data property can be assigned a new value directly.

207. Why does writable matter in production code?

Answer

Writable determines whether a data property can be assigned a new value directly. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

208. What are common interview traps with writable?

Answer

Writable determines whether a data property can be assigned a new value directly. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

209. What is configurable?

Answer

Configurable determines whether a property can be deleted or its descriptor changed. Non-configurable properties are deliberately harder to alter.

210. How does configurable work?

Answer

Configurable determines whether a property can be deleted or its descriptor changed. Non-configurable properties are deliberately harder to alter. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

211. What is object.freeze?

Answer

Object.freeze prevents extensions and makes existing own data properties non-writable and non-configurable. It is shallow: nested objects can still mutate.

212. Why does object.freeze matter in production code?

Answer

Object.freeze prevents extensions and makes existing own data properties non-writable and non-configurable. It is shallow: nested objects can still mutate. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

213. What is object.seal?

Answer

Object.seal prevents adding or removing own properties and makes them non-configurable, but existing writable data properties may still change.

214. How does object.seal work?

Answer

Object.seal prevents adding or removing own properties and makes them non-configurable, but existing writable data properties may still change. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

215. What are common interview traps with object.seal?

Answer

Object.seal prevents adding or removing own properties and makes them non-configurable, but existing writable data properties may still change. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

216. What is preventextensions?

Answer

Object.preventExtensions stops new own properties from being added but does not by itself make existing properties non-writable or non-configurable.

217. Why does preventextensions matter in production code?

Answer

Object.preventExtensions stops new own properties from being added but does not by itself make existing properties non-writable or non-configurable. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

218. What is shallow copy?

Answer

A shallow copy creates a new outer container but keeps the same nested object references. Spread syntax and Object.assign are shallow copies.

219. How does shallow copy work?

Answer

A shallow copy creates a new outer container but keeps the same nested object references. Spread syntax and Object.assign are shallow copies. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

220. What is deep copy?

Answer

A deep copy recursively duplicates the relevant object graph so nested mutable objects are independent. No single clone strategy works for every JavaScript value.

221. Why does deep copy matter in production code?

Answer

A deep copy recursively duplicates the relevant object graph so nested mutable objects are independent. No single clone strategy works for every JavaScript value. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

222. What are common interview traps with deep copy?

Answer

A deep copy recursively duplicates the relevant object graph so nested mutable objects are independent. No single clone strategy works for every JavaScript value. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

223. What is structuredclone?

Answer

structuredClone performs the platform structured clone algorithm for many built-in data types and supports circular references. It does not clone functions or DOM nodes.

224. How does structuredclone work?

Answer

structuredClone performs the platform structured clone algorithm for many built-in data types and supports circular references. It does not clone functions or DOM nodes. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

9. Arrays and Collections

33 interview prompts

225. What is array?

Answer

A JavaScript array is an indexed object with a length property and special array methods. Engines may optimize dense arrays, but arrays are not the same abstraction as fixed-size typed arrays.

226. How does array work?

Answer

A JavaScript array is an indexed object with a length property and special array methods. Engines may optimize dense arrays, but arrays are not the same abstraction as fixed-size typed arrays. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

227. What are common interview traps with array?

Answer

A JavaScript array is an indexed object with a length property and special array methods. Engines may optimize dense arrays, but arrays are not the same abstraction as fixed-size typed arrays. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

228. What is array-like?

Answer

An array-like object has indexed properties and usually a length property but does not necessarily implement array methods or the iterable protocol.

229. Why does array-like matter in production code?

Answer

An array-like object has indexed properties and usually a length property but does not necessarily implement array methods or the iterable protocol. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

230. What is sparse array?

Answer

A sparse array has missing indexes (holes). A hole is not the same as an explicit undefined value, and different array methods treat holes differently.

231. How does sparse array work?

Answer

A sparse array has missing indexes (holes). A hole is not the same as an explicit undefined value, and different array methods treat holes differently. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

232. What is map?

Answer

Array.prototype.map creates a new array containing the callback result for each visited element. It does not mutate the original array unless the callback itself mutates something.

233. Why does map matter in production code?

Answer

Array.prototype.map creates a new array containing the callback result for each visited element. It does not mutate the original array unless the callback itself mutates something. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

234. What are common interview traps with map?

Answer

Array.prototype.map creates a new array containing the callback result for each visited element. It does not mutate the original array unless the callback itself mutates something. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

235. What is foreach?

Answer

Array.prototype.forEach calls a callback for each visited element and returns undefined. It is for side effects rather than producing a transformed array.

236. How does foreach work?

Answer

Array.prototype.forEach calls a callback for each visited element and returns undefined. It is for side effects rather than producing a transformed array. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

237. What is filter?

Answer

filter creates a new array containing elements for which the predicate returns a truthy result.

238. Why does filter matter in production code?

Answer

filter creates a new array containing elements for which the predicate returns a truthy result. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

239. What is find?

Answer

find returns the first element satisfying a predicate, or undefined if none does. It stops once a match is found.

240. How does find work?

Answer

find returns the first element satisfying a predicate, or undefined if none does. It stops once a match is found. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

241. What are common interview traps with find?

Answer

find returns the first element satisfying a predicate, or undefined if none does. It stops once a match is found. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

242. What is reduce?

Answer

reduce folds an array into one accumulated result using a callback. It is powerful but can become hard to read when overused for unrelated transformations.

243. Why does reduce matter in production code?

Answer

reduce folds an array into one accumulated result using a callback. It is powerful but can become hard to read when overused for unrelated transformations. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

244. What is slice?

Answer

slice returns a shallow copy of a selected range and does not mutate the original array.

245. How does slice work?

Answer

slice returns a shallow copy of a selected range and does not mutate the original array. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

246. What is splice?

Answer

splice changes an array in place by removing, inserting, or replacing elements.

247. Why does splice matter in production code?

Answer

splice changes an array in place by removing, inserting, or replacing elements. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

248. What are common interview traps with splice?

Answer

splice changes an array in place by removing, inserting, or replacing elements. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

249. What is set?

Answer

Set stores unique values using SameValueZero equality. It is useful for membership checks and deduplication.

250. How does set work?

Answer

Set stores unique values using SameValueZero equality. It is useful for membership checks and deduplication. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

251. What is map object?

Answer

Map stores key-value pairs with keys of any JavaScript value and has predictable insertion-order iteration. It is often clearer than an object for dictionary-like data with non-string keys.

252. Why does map object matter in production code?

Answer

Map stores key-value pairs with keys of any JavaScript value and has predictable insertion-order iteration. It is often clearer than an object for dictionary-like data with non-string keys. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

253. What is weakmap?

Answer

WeakMap maps object or non-registered-symbol keys to values without preventing key garbage collection. It is useful for metadata tied to object lifetime.

254. How does weakmap work?

Answer

WeakMap maps object or non-registered-symbol keys to values without preventing key garbage collection. It is useful for metadata tied to object lifetime. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

255. What are common interview traps with weakmap?

Answer

WeakMap maps object or non-registered-symbol keys to values without preventing key garbage collection. It is useful for metadata tied to object lifetime. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

256. What is weakset?

Answer

WeakSet stores object or non-registered-symbol identities without preventing garbage collection. It is useful for tracking object membership without owning those objects.

257. Why does weakset matter in production code?

Answer

WeakSet stores object or non-registered-symbol identities without preventing garbage collection. It is useful for tracking object membership without owning those objects. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

10. Promise Internals

34 interview prompts

258. What is promise?

Answer

A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final.

259. How does promise work?

Answer

A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

260. What are common interview traps with promise?

Answer

A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

261. What is promise state?

Answer

A Promise starts pending and becomes fulfilled or rejected. Once settled, its state and result are immutable.

262. Why does promise state matter in production code?

Answer

A Promise starts pending and becomes fulfilled or rejected. Once settled, its state and result are immutable. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

263. What is promise resolution?

Answer

Promise resolution adopts the state of another Promise or thenable when necessary. Resolution is the broader process; fulfillment is one possible final state.

264. How does promise resolution work?

Answer

Promise resolution adopts the state of another Promise or thenable when necessary. Resolution is the broader process; fulfillment is one possible final state. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

265. What is thenable?

Answer

A thenable is an object with a callable then property. Promise resolution assimilates compatible thenables so they behave like Promises in chains.

266. Why does thenable matter in production code?

Answer

A thenable is an object with a callable then property. Promise resolution assimilates compatible thenables so they behave like Promises in chains. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

267. What are common interview traps with thenable?

Answer

A thenable is an object with a callable then property. Promise resolution assimilates compatible thenables so they behave like Promises in chains. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

268. What is promise chaining?

Answer

then, catch, and finally return new Promises, creating a chain. The next Promise adopts the callback result, or becomes rejected if the callback throws.

269. How does promise chaining work?

Answer

then, catch, and finally return new Promises, creating a chain. The next Promise adopts the callback result, or becomes rejected if the callback throws. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

270. What is promise.all?

Answer

Promise.all waits for all inputs to fulfill and preserves input order in its result array. It rejects as soon as the combined operation is known to have failed, but it does not cancel the underlying operations.

271. Why does promise.all matter in production code?

Answer

Promise.all waits for all inputs to fulfill and preserves input order in its result array. It rejects as soon as the combined operation is known to have failed, but it does not cancel the underlying operations. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

272. What is promise.allsettled?

Answer

Promise.allSettled waits for every input and returns an array describing each fulfilled or rejected outcome.

273. How does promise.allsettled work?

Answer

Promise.allSettled waits for every input and returns an array describing each fulfilled or rejected outcome. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

274. What are common interview traps with promise.allsettled?

Answer

Promise.allSettled waits for every input and returns an array describing each fulfilled or rejected outcome. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

275. What is promise.race?

Answer

Promise.race settles as soon as the first input Promise settles, whether fulfilled or rejected. It does not cancel the other operations.

276. Why does promise.race matter in production code?

Answer

Promise.race settles as soon as the first input Promise settles, whether fulfilled or rejected. It does not cancel the other operations. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

277. What is promise.any?

Answer

Promise.any fulfills when the first input fulfills and rejects only when all inputs reject, using AggregateError for the combined failure.

278. How does promise.any work?

Answer

Promise.any fulfills when the first input fulfills and rejects only when all inputs reject, using AggregateError for the combined failure. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

279. What is aggregateerror?

Answer

AggregateError is an Error type used when multiple underlying errors are represented together, notably by Promise.any().

280. Why does aggregateerror matter in production code?

Answer

AggregateError is an Error type used when multiple underlying errors are represented together, notably by Promise.any(). Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

281. What are common interview traps with aggregateerror?

Answer

AggregateError is an Error type used when multiple underlying errors are represented together, notably by Promise.any(). Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

282. What happens when a then callback returns a value?

Answer

A callback is a function passed to another function or API so it can be invoked later or during a particular operation.

283. What happens when a then callback returns a Promise?

Answer

A callback is a function passed to another function or API so it can be invoked later or during a particular operation.

284. What happens when a then callback throws?

Answer

A callback is a function passed to another function or API so it can be invoked later or during a particular operation.

285. What happens when catch throws?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

286. Does finally receive the resolved value?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

287. Why does then return another Promise?

Answer

A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

288. Can a Promise be cancelled?

Answer

A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final.

289. Why does Promise.all not cancel remaining work?

Answer

Promise.all waits for all inputs to fulfill and preserves input order in its result array. It rejects as soon as the combined operation is known to have failed, but it does not cancel the underlying operations. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

290. Difference between Promise.resolve and new Promise?

Answer

Promise.resolve and new Promise solve related problems but differ in semantics and trade-offs. A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final. A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final.

291. How would you implement Promise.all?

Answer

Promise.all waits for all inputs to fulfill and preserves input order in its result array. It rejects as soon as the combined operation is known to have failed, but it does not cancel the underlying operations. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

11. async/await

10 interview prompts

292. What is async function?

Answer

An async function always returns a Promise. Returning a value fulfills that Promise; throwing produces a rejected Promise.

293. How does async function work?

Answer

An async function always returns a Promise. Returning a value fulfills that Promise; throwing produces a rejected Promise. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

294. What are common interview traps with async function?

Answer

An async function always returns a Promise. Returning a value fulfills that Promise; throwing produces a rejected Promise. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

295. What is await?

Answer

await pauses the current async function until the awaited value is settled, then resumes through Promise continuation semantics. It does not block the JavaScript thread.

296. Why does await matter in production code?

Answer

await pauses the current async function until the awaited value is settled, then resumes through Promise continuation semantics. It does not block the JavaScript thread. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

297. What is try/catch?

Answer

try/catch catches synchronous exceptions in the same call path. It also catches an exception thrown synchronously inside an async function before that function returns its rejected Promise, but it does not automatically catch unrelated later Promise rejections.

298. Why does try/catch matter in production code?

Answer

try/catch catches synchronous exceptions in the same call path. It also catches an exception thrown synchronously inside an async function before that function returns its rejected Promise, but it does not automatically catch unrelated later Promise rejections. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

299. What are common interview traps with try/catch?

Answer

try/catch catches synchronous exceptions in the same call path. It also catches an exception thrown synchronously inside an async function before that function returns its rejected Promise, but it does not automatically catch unrelated later Promise rejections. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

300. What is unhandled rejection?

Answer

An unhandled rejection is a rejected Promise for which no rejection handler is attached in time. Browsers and Node report these through host-specific global mechanisms.

301. How does unhandled rejection work?

Answer

An unhandled rejection is a rejected Promise for which no rejection handler is attached in time. Browsers and Node report these through host-specific global mechanisms. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

12. Event Loop and Scheduling

24 interview prompts

302. What is event loop?

Answer

The event loop coordinates JavaScript execution with queued tasks, microtasks, rendering opportunities, and host APIs. JavaScript on a given thread executes one stack at a time.

303. How does event loop work?

Answer

The event loop coordinates JavaScript execution with queued tasks, microtasks, rendering opportunities, and host APIs. JavaScript on a given thread executes one stack at a time. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

304. What are common interview traps with event loop?

Answer

The event loop coordinates JavaScript execution with queued tasks, microtasks, rendering opportunities, and host APIs. JavaScript on a given thread executes one stack at a time. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

305. What is microtask?

Answer

A microtask is a job processed after the current JavaScript stack clears and before the event loop proceeds to a later task. Promise reactions and queueMicrotask are common sources.

306. How does microtask work?

Answer

A microtask is a job processed after the current JavaScript stack clears and before the event loop proceeds to a later task. Promise reactions and queueMicrotask are common sources. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

307. What is macrotask?

Answer

In browser interview terminology, macrotask commonly means a task such as a timer callback, user interaction, or network event callback. The browser runs tasks one at a time and drains microtasks between tasks.

308. Why does macrotask matter in production code?

Answer

In browser interview terminology, macrotask commonly means a task such as a timer callback, user interaction, or network event callback. The browser runs tasks one at a time and drains microtasks between tasks. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

309. What are common interview traps with macrotask?

Answer

In browser interview terminology, macrotask commonly means a task such as a timer callback, user interaction, or network event callback. The browser runs tasks one at a time and drains microtasks between tasks. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

310. What is task?

Answer

The HTML event loop schedules tasks from sources such as timers and user interaction. After a task finishes, the browser performs the required microtask checkpoint before moving on.

311. How does task work?

Answer

The HTML event loop schedules tasks from sources such as timers and user interaction. After a task finishes, the browser performs the required microtask checkpoint before moving on. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

312. What is run-to-completion?

Answer

Run-to-completion means once JavaScript begins executing a task, that synchronous code runs until it finishes before another task can execute on the same thread.

313. Why does run-to-completion matter in production code?

Answer

Run-to-completion means once JavaScript begins executing a task, that synchronous code runs until it finishes before another task can execute on the same thread. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

314. What is event-loop starvation?

Answer

Starvation occurs when continuously queued microtasks or long-running synchronous work prevent other tasks or rendering opportunities from getting a turn.

315. How does event-loop starvation work?

Answer

Starvation occurs when continuously queued microtasks or long-running synchronous work prevent other tasks or rendering opportunities from getting a turn. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

316. What are common interview traps with event-loop starvation?

Answer

Starvation occurs when continuously queued microtasks or long-running synchronous work prevent other tasks or rendering opportunities from getting a turn. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

317. Does setTimeout(fn, 0) run immediately?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

318. Why do Promise callbacks usually run before timers?

Answer

A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

319. When are microtasks drained?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

320. Can microtasks starve rendering?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

321. Can microtasks starve timers?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

322. What happens when a microtask queues another microtask?

Answer

A task queue holds pending work and controls the order and concurrency with which tasks execute. It is a foundation for schedulers and worker pools.

323. What happens when a timer queues a Promise callback?

Answer

A callback is a function passed to another function or API so it can be invoked later or during a particular operation.

324. What happens when Promise.resolve().then queues setTimeout?

Answer

A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final.

325. What determines which task runs next?

Answer

The HTML event loop schedules tasks from sources such as timers and user interaction. After a task finishes, the browser performs the required microtask checkpoint before moving on.

13. Fetch, Network and Cancellation

36 interview prompts

326. What is fetch?

Answer

fetch returns a Promise that fulfills with a Response when the request receives a response, even for HTTP error statuses such as 404. It rejects for failures such as a network error or an abort.

327. How does fetch work?

Answer

fetch returns a Promise that fulfills with a Response when the request receives a response, even for HTTP error statuses such as 404. It rejects for failures such as a network error or an abort. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

328. What are common interview traps with fetch?

Answer

fetch returns a Promise that fulfills with a Response when the request receives a response, even for HTTP error statuses such as 404. It rejects for failures such as a network error or an abort. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

329. What is response.json?

Answer

Response.json() asynchronously reads and parses the response body, returning a Promise for the parsed JavaScript value.

330. Why does response.json matter in production code?

Answer

Response.json() asynchronously reads and parses the response body, returning a Promise for the parsed JavaScript value. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

331. What is network error?

Answer

A network error is a failure to complete the fetch operation, such as DNS failure, connection failure, or abort. HTTP 4xx/5xx responses are normally still successful fetches and must be checked separately with response.ok/status.

332. How does network error work?

Answer

A network error is a failure to complete the fetch operation, such as DNS failure, connection failure, or abort. HTTP 4xx/5xx responses are normally still successful fetches and must be checked separately with response.ok/status. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

333. What is abortcontroller?

Answer

AbortController provides an AbortSignal that APIs such as fetch and addEventListener can observe. Calling abort() signals cancellation to those APIs; it does not magically undo work already completed on a server.

334. Why does abortcontroller matter in production code?

Answer

AbortController provides an AbortSignal that APIs such as fetch and addEventListener can observe. Calling abort() signals cancellation to those APIs; it does not magically undo work already completed on a server. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

335. What are common interview traps with abortcontroller?

Answer

AbortController provides an AbortSignal that APIs such as fetch and addEventListener can observe. Calling abort() signals cancellation to those APIs; it does not magically undo work already completed on a server. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

336. What is request cancellation?

Answer

Cancellation asks an API to stop or ignore work that is no longer needed. With fetch, AbortController is the standard browser mechanism for signaling cancellation.

337. How does request cancellation work?

Answer

Cancellation asks an API to stop or ignore work that is no longer needed. With fetch, AbortController is the standard browser mechanism for signaling cancellation. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

338. What is race condition?

Answer

A race condition occurs when correctness depends on timing or interleaving of concurrent operations. In UI code, an older response can overwrite newer state unless requests are canceled, sequenced, or ignored when stale.

339. Why does race condition matter in production code?

Answer

A race condition occurs when correctness depends on timing or interleaving of concurrent operations. In UI code, an older response can overwrite newer state unless requests are canceled, sequenced, or ignored when stale. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

340. What is retry?

Answer

Retry repeats a failed operation according to a policy. Production retry logic should bound attempts, distinguish retryable failures, and usually use exponential backoff with jitter.

341. How does retry work?

Answer

Retry repeats a failed operation according to a policy. Production retry logic should bound attempts, distinguish retryable failures, and usually use exponential backoff with jitter. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

342. What are common interview traps with retry?

Answer

Retry repeats a failed operation according to a policy. Production retry logic should bound attempts, distinguish retryable failures, and usually use exponential backoff with jitter. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

343. What is exponential backoff?

Answer

Exponential backoff increases the delay between retries, reducing load on a failing dependency. Jitter randomizes those delays to avoid synchronized retry spikes.

344. Why does exponential backoff matter in production code?

Answer

Exponential backoff increases the delay between retries, reducing load on a failing dependency. Jitter randomizes those delays to avoid synchronized retry spikes. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

345. What is polling?

Answer

Polling repeatedly requests or checks for updated state at a chosen interval. It is simple but can waste traffic or add latency compared with push-based approaches.

346. How does polling work?

Answer

Polling repeatedly requests or checks for updated state at a chosen interval. It is simple but can waste traffic or add latency compared with push-based approaches. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

347. What is websocket?

Answer

WebSocket provides a persistent, bidirectional connection between client and server after an HTTP-based handshake, useful for low-latency interactive updates.

348. Why does websocket matter in production code?

Answer

WebSocket provides a persistent, bidirectional connection between client and server after an HTTP-based handshake, useful for low-latency interactive updates. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

349. What are common interview traps with websocket?

Answer

WebSocket provides a persistent, bidirectional connection between client and server after an HTTP-based handshake, useful for low-latency interactive updates. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

350. What is server-sent events?

Answer

Server-Sent Events provide a one-way stream from server to browser over HTTP using EventSource. The server pushes events; the browser sends separate requests back.

351. How does server-sent events work?

Answer

Server-Sent Events provide a one-way stream from server to browser over HTTP using EventSource. The server pushes events; the browser sends separate requests back. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

352. Does fetch reject on HTTP 404?

Answer

fetch returns a Promise that fulfills with a Response when the request receives a response, even for HTTP error statuses such as 404. It rejects for failures such as a network error or an abort.

353. Why should response.ok be checked?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

354. Does fetch block the call stack?

Answer

The call stack tracks currently executing function calls. JavaScript execution is run-to-completion on a given thread: one stack frame runs until it returns or throws.

355. Who performs the actual network operation?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

356. When does a fetch Promise settle?

Answer

A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final.

357. What happens after await fetch?

Answer

await pauses the current async function until the awaited value is settled, then resumes through Promise continuation semantics. It does not block the JavaScript thread.

358. How would you implement fetch timeout with AbortController?

Answer

AbortController provides an AbortSignal that APIs such as fetch and addEventListener can observe. Calling abort() signals cancellation to those APIs; it does not magically undo work already completed on a server. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

359. Does aborting fetch undo work already done on the server?

Answer

fetch returns a Promise that fulfills with a Response when the request receives a response, even for HTTP error statuses such as 404. It rejects for failures such as a network error or an abort.

360. How do you avoid stale search results?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

361. How would you retry with exponential backoff?

Answer

Exponential backoff increases the delay between retries, reducing load on a failing dependency. Jitter randomizes those delays to avoid synchronized retry spikes. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

14. DOM and Browser Events

28 interview prompts

362. What is dom?

Answer

The DOM is the browser API representation of a document as nodes and objects. It is not the JavaScript language itself; JavaScript interacts with the DOM through host APIs.

363. How does dom work?

Answer

The DOM is the browser API representation of a document as nodes and objects. It is not the JavaScript language itself; JavaScript interacts with the DOM through host APIs. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

364. What are common interview traps with dom?

Answer

The DOM is the browser API representation of a document as nodes and objects. It is not the JavaScript language itself; JavaScript interacts with the DOM through host APIs. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

365. What is bom?

Answer

The Browser Object Model is the informal term for browser-provided objects such as window, location, history, navigator, and screen that sit around the document model.

366. Why does bom matter in production code?

Answer

The Browser Object Model is the informal term for browser-provided objects such as window, location, history, navigator, and screen that sit around the document model. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

367. What is event bubbling?

Answer

Bubbling propagates an event from its target upward through ancestors after the target phase.

368. How does event bubbling work?

Answer

Bubbling propagates an event from its target upward through ancestors after the target phase. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

369. What is event capturing?

Answer

Capturing propagates an event from ancestors down toward the target before the target phase.

370. Why does event capturing matter in production code?

Answer

Capturing propagates an event from ancestors down toward the target before the target phase. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

371. What are common interview traps with event capturing?

Answer

Capturing propagates an event from ancestors down toward the target before the target phase. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

372. What is event propagation?

Answer

Propagation is the event flow through capturing, target, and bubbling phases. preventDefault affects the default action; stopPropagation affects propagation.

373. How does event propagation work?

Answer

Propagation is the event flow through capturing, target, and bubbling phases. preventDefault affects the default action; stopPropagation affects propagation. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

374. What is event delegation?

Answer

Event delegation attaches one listener to a common ancestor and uses the event target path to handle many descendant interactions. It reduces listener count and works well with dynamic content.

375. Why does event delegation matter in production code?

Answer

Event delegation attaches one listener to a common ancestor and uses the event target path to handle many descendant interactions. It reduces listener count and works well with dynamic content. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

376. What is preventdefault?

Answer

preventDefault cancels the browser default action for a cancelable event, such as following a link or submitting a form. It does not stop propagation by itself.

377. How does preventdefault work?

Answer

preventDefault cancels the browser default action for a cancelable event, such as following a link or submitting a form. It does not stop propagation by itself. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

378. What are common interview traps with preventdefault?

Answer

preventDefault cancels the browser default action for a cancelable event, such as following a link or submitting a form. It does not stop propagation by itself. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

379. What is stoppropagation?

Answer

stopPropagation prevents the event from continuing to other nodes during propagation, but it does not stop other listeners on the same node.

380. Why does stoppropagation matter in production code?

Answer

stopPropagation prevents the event from continuing to other nodes during propagation, but it does not stop other listeners on the same node. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

381. What is stopimmediatepropagation?

Answer

stopImmediatePropagation stops propagation and also prevents later listeners on the same target from running.

382. How does stopimmediatepropagation work?

Answer

stopImmediatePropagation stops propagation and also prevents later listeners on the same target from running. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

383. What is event target?

Answer

event.target is the original target of the dispatched event, while event.currentTarget is the element whose listener is currently executing.

384. Why does event target matter in production code?

Answer

event.target is the original target of the dispatched event, while event.currentTarget is the element whose listener is currently executing. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

385. What are common interview traps with event target?

Answer

event.target is the original target of the dispatched event, while event.currentTarget is the element whose listener is currently executing. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

386. What is passive listener?

Answer

A passive event listener promises not to call preventDefault. Browsers can use that guarantee to improve scrolling performance for events such as touchmove and wheel.

387. How does passive listener work?

Answer

A passive event listener promises not to call preventDefault. Browsers can use that guarantee to improve scrolling performance for events such as touchmove and wheel. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

388. What is custom event?

Answer

CustomEvent lets application code dispatch named events with optional custom data in detail. It is useful for loosely coupled DOM-based communication.

389. Why does custom event matter in production code?

Answer

CustomEvent lets application code dispatch named events with optional custom data in detail. It is useful for loosely coupled DOM-based communication. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

15. Browser Rendering

32 interview prompts

390. What is rendering pipeline?

Answer

A simplified browser rendering pipeline is style calculation, layout, paint, and compositing, with JavaScript and DOM changes potentially causing work at several stages.

391. How does rendering pipeline work?

Answer

A simplified browser rendering pipeline is style calculation, layout, paint, and compositing, with JavaScript and DOM changes potentially causing work at several stages. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

392. What are common interview traps with rendering pipeline?

Answer

A simplified browser rendering pipeline is style calculation, layout, paint, and compositing, with JavaScript and DOM changes potentially causing work at several stages. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

393. What is reflow?

Answer

Reflow, more precisely layout, computes geometry and positions. DOM or style changes that affect geometry can trigger layout work.

394. Why does reflow matter in production code?

Answer

Reflow, more precisely layout, computes geometry and positions. DOM or style changes that affect geometry can trigger layout work. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

395. What is repaint?

Answer

Repaint is redrawing pixels after visual properties change without necessarily requiring new layout geometry.

396. How does repaint work?

Answer

Repaint is redrawing pixels after visual properties change without necessarily requiring new layout geometry. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

397. What is layout thrashing?

Answer

Layout thrashing occurs when code repeatedly writes style/layout-affecting changes and then synchronously reads layout information, forcing the browser to recalculate layout repeatedly.

398. Why does layout thrashing matter in production code?

Answer

Layout thrashing occurs when code repeatedly writes style/layout-affecting changes and then synchronously reads layout information, forcing the browser to recalculate layout repeatedly. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

399. What are common interview traps with layout thrashing?

Answer

Layout thrashing occurs when code repeatedly writes style/layout-affecting changes and then synchronously reads layout information, forcing the browser to recalculate layout repeatedly. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

400. What is forced synchronous layout?

Answer

Reading certain geometry values after invalidating layout can force the browser to perform layout immediately so the script receives up-to-date values.

401. How does forced synchronous layout work?

Answer

Reading certain geometry values after invalidating layout can force the browser to perform layout immediately so the script receives up-to-date values. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

402. What is requestanimationframe?

Answer

requestAnimationFrame schedules a callback before a browser paint opportunity, making it appropriate for visual updates synchronized with display refresh.

403. Why does requestanimationframe matter in production code?

Answer

requestAnimationFrame schedules a callback before a browser paint opportunity, making it appropriate for visual updates synchronized with display refresh. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

404. What is requestidlecallback?

Answer

requestIdleCallback schedules lower-priority work during periods the browser considers idle. It should not be used for latency-critical work because execution timing is not guaranteed.

405. How does requestidlecallback work?

Answer

requestIdleCallback schedules lower-priority work during periods the browser considers idle. It should not be used for latency-critical work because execution timing is not guaranteed. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

406. What are common interview traps with requestidlecallback?

Answer

requestIdleCallback schedules lower-priority work during periods the browser considers idle. It should not be used for latency-critical work because execution timing is not guaranteed. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

407. What is main thread?

Answer

The main thread is the browser thread commonly responsible for JavaScript, DOM work, style, and layout coordination. Long tasks on it can block input and rendering.

408. Why does main thread matter in production code?

Answer

The main thread is the browser thread commonly responsible for JavaScript, DOM work, style, and layout coordination. Long tasks on it can block input and rendering. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

409. What is long task?

Answer

A long task is a main-thread task long enough to noticeably delay responsiveness, commonly discussed around the 50 ms threshold for performance tooling.

410. How does long task work?

Answer

A long task is a main-thread task long enough to noticeably delay responsiveness, commonly discussed around the 50 ms threshold for performance tooling. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

411. What is frame budget?

Answer

At 60 Hz a frame arrives about every 16.67 ms. JavaScript, style, layout, paint, and other work must fit within the available time to maintain smoothness.

412. Why does frame budget matter in production code?

Answer

At 60 Hz a frame arrives about every 16.67 ms. JavaScript, style, layout, paint, and other work must fit within the available time to maintain smoothness. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

413. What are common interview traps with frame budget?

Answer

At 60 Hz a frame arrives about every 16.67 ms. JavaScript, style, layout, paint, and other work must fit within the available time to maintain smoothness. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

414. What causes reflow?

Answer

Reflow, more precisely layout, computes geometry and positions. DOM or style changes that affect geometry can trigger layout work.

415. What causes repaint?

Answer

Repaint is redrawing pixels after visual properties change without necessarily requiring new layout geometry.

416. Why is requestAnimationFrame preferred for animation?

Answer

requestAnimationFrame schedules a callback before a browser paint opportunity, making it appropriate for visual updates synchronized with display refresh. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

417. Can JavaScript prevent the browser from rendering?

Answer

JavaScript is a high-level, dynamically typed language standardized by ECMAScript. Browsers and runtimes such as Node.js execute it through an engine plus host-provided APIs.

418. Why can long tasks hurt input latency?

Answer

A long task is a main-thread task long enough to noticeably delay responsiveness, commonly discussed around the 50 ms threshold for performance tooling. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

419. What is a compositor layer?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

420. Why can transforms be cheaper than layout changes?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

421. How do you profile rendering performance?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

16. JavaScript Engines and JIT

26 interview prompts

422. What is javascript engine?

Answer

A JavaScript engine parses and executes ECMAScript code and may compile hot code for performance. V8, SpiderMonkey, and JavaScriptCore are examples.

423. How does javascript engine work?

Answer

A JavaScript engine parses and executes ECMAScript code and may compile hot code for performance. V8, SpiderMonkey, and JavaScriptCore are examples. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

424. What are common interview traps with javascript engine?

Answer

A JavaScript engine parses and executes ECMAScript code and may compile hot code for performance. V8, SpiderMonkey, and JavaScriptCore are examples. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

425. What is v8?

Answer

V8 is Google's JavaScript and WebAssembly engine used by Chrome and Node.js. It performs parsing, compilation, optimization, garbage collection, and execution.

426. Why does v8 matter in production code?

Answer

V8 is Google's JavaScript and WebAssembly engine used by Chrome and Node.js. It performs parsing, compilation, optimization, garbage collection, and execution. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

427. What is ast?

Answer

An abstract syntax tree represents parsed program structure. Compilers and tools can use it for analysis, transformation, optimization, and code generation.

428. How does ast work?

Answer

An abstract syntax tree represents parsed program structure. Compilers and tools can use it for analysis, transformation, optimization, and code generation. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

429. What is bytecode?

Answer

Bytecode is an intermediate instruction representation used by many engines between parsing and native machine-code execution or optimization.

430. Why does bytecode matter in production code?

Answer

Bytecode is an intermediate instruction representation used by many engines between parsing and native machine-code execution or optimization. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

431. What are common interview traps with bytecode?

Answer

Bytecode is an intermediate instruction representation used by many engines between parsing and native machine-code execution or optimization. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

432. What is jit?

Answer

JIT compilation means compiling code at runtime based on observed behavior. Modern engines can interpret or baseline-compile code first, then optimize hot paths speculatively.

433. How does jit work?

Answer

JIT compilation means compiling code at runtime based on observed behavior. Modern engines can interpret or baseline-compile code first, then optimize hot paths speculatively. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

434. What is deoptimization?

Answer

Deoptimization is falling back from optimized machine code when assumptions used by the optimizer are no longer valid, such as a changed type or object shape.

435. Why does deoptimization matter in production code?

Answer

Deoptimization is falling back from optimized machine code when assumptions used by the optimizer are no longer valid, such as a changed type or object shape. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

436. What is hidden class?

Answer

A hidden class is an engine-internal representation of an object's shape. Consistent property creation order can let engines optimize property access.

437. How does hidden class work?

Answer

A hidden class is an engine-internal representation of an object's shape. Consistent property creation order can let engines optimize property access. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

438. What are common interview traps with hidden class?

Answer

A hidden class is an engine-internal representation of an object's shape. Consistent property creation order can let engines optimize property access. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

439. What is inline cache?

Answer

An inline cache stores observed information about property or call-site behavior so repeated operations can avoid repeating generic lookup work. Stable object shapes improve its effectiveness.

440. Why does inline cache matter in production code?

Answer

An inline cache stores observed information about property or call-site behavior so repeated operations can avoid repeating generic lookup work. Stable object shapes improve its effectiveness. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

441. What is monomorphic?

Answer

A monomorphic call site consistently sees one relevant object shape or type pattern, making aggressive specialization easier for a JavaScript engine.

442. How does monomorphic work?

Answer

A monomorphic call site consistently sees one relevant object shape or type pattern, making aggressive specialization easier for a JavaScript engine. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

443. What is polymorphic?

Answer

A polymorphic call site sees a small number of recurring shapes or type patterns. Engines can often optimize several common cases.

444. Why does polymorphic matter in production code?

Answer

A polymorphic call site sees a small number of recurring shapes or type patterns. Engines can often optimize several common cases. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

445. What are common interview traps with polymorphic?

Answer

A polymorphic call site sees a small number of recurring shapes or type patterns. Engines can often optimize several common cases. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

446. What is megamorphic?

Answer

A megamorphic site sees many different shapes or patterns, making specialized inline caching less effective and often pushing execution toward generic lookup paths.

447. How does megamorphic work?

Answer

A megamorphic site sees many different shapes or patterns, making specialized inline caching less effective and often pushing execution toward generic lookup paths. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

17. Garbage Collection and Memory

24 interview prompts

448. What is garbage collection?

Answer

Garbage collection reclaims memory for objects that are no longer reachable. Engines use multiple strategies, commonly including generational collection and tracing.

449. How does garbage collection work?

Answer

Garbage collection reclaims memory for objects that are no longer reachable. Engines use multiple strategies, commonly including generational collection and tracing. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

450. What are common interview traps with garbage collection?

Answer

Garbage collection reclaims memory for objects that are no longer reachable. Engines use multiple strategies, commonly including generational collection and tracing. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

451. What is mark-and-sweep?

Answer

Mark-and-sweep traces reachable objects from roots, marks them, then reclaims unmarked memory. Modern collectors add generations, incremental work, compaction, and concurrency.

452. Why does mark-and-sweep matter in production code?

Answer

Mark-and-sweep traces reachable objects from roots, marks them, then reclaims unmarked memory. Modern collectors add generations, incremental work, compaction, and concurrency. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

453. What is generational gc?

Answer

Generational garbage collection exploits the observation that many objects die young. Collectors often separate young and old generations to make collection more efficient.

454. How does generational gc work?

Answer

Generational garbage collection exploits the observation that many objects die young. Collectors often separate young and old generations to make collection more efficient. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

455. What is heap snapshot?

Answer

A heap snapshot records the heap graph so you can inspect retained objects and reference paths. Comparing snapshots over time is a common way to diagnose leaks.

456. How does heap snapshot work?

Answer

A heap snapshot records the heap graph so you can inspect retained objects and reference paths. Comparing snapshots over time is a common way to diagnose leaks. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

457. What is weakref?

Answer

WeakRef provides a non-owning reference to an object that may be garbage-collected. Code must tolerate the target disappearing at any time; it is not suitable for deterministic resource management.

458. Why does weakref matter in production code?

Answer

WeakRef provides a non-owning reference to an object that may be garbage-collected. Code must tolerate the target disappearing at any time; it is not suitable for deterministic resource management. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

459. What is finalizationregistry?

Answer

FinalizationRegistry lets code register cleanup callbacks associated with garbage-collection of targets, but timing is nondeterministic and should not control correctness-critical cleanup.

460. How does finalizationregistry work?

Answer

FinalizationRegistry lets code register cleanup callbacks associated with garbage-collection of targets, but timing is nondeterministic and should not control correctness-critical cleanup. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

461. What are common interview traps with finalizationregistry?

Answer

FinalizationRegistry lets code register cleanup callbacks associated with garbage-collection of targets, but timing is nondeterministic and should not control correctness-critical cleanup. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

462. How does a garbage collector know an object is no longer needed?

Answer

An object is a collection of properties identified by keys. Property values can be data, accessors, or functions, and the object can delegate through a prototype. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

463. What is reachable memory?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

464. Can closures keep objects alive?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

465. How do event listeners cause memory leaks?

Answer

A JavaScript memory leak occurs when objects remain reachable even though the application no longer needs them, so garbage collection cannot reclaim them. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

466. How do timers cause memory leaks?

Answer

A JavaScript memory leak occurs when objects remain reachable even though the application no longer needs them, so garbage collection cannot reclaim them. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

467. What is a retained object?

Answer

An object is a collection of properties identified by keys. Property values can be data, accessors, or functions, and the object can delegate through a prototype.

468. What is a retainer path?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

469. Why can WeakMap help with object metadata?

Answer

WeakMap maps object or non-registered-symbol keys to values without preventing key garbage collection. It is useful for metadata tied to object lifetime. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

470. Why is WeakRef not deterministic?

Answer

WeakRef provides a non-owning reference to an object that may be garbage-collected. Code must tolerate the target disappearing at any time; it is not suitable for deterministic resource management. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

471. How do you find memory leaks in Chrome DevTools?

Answer

A JavaScript memory leak occurs when objects remain reachable even though the application no longer needs them, so garbage collection cannot reclaim them. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

18. Modules and Loading

29 interview prompts

472. What is es modules?

Answer

ES modules use static import/export syntax, have module scope, and expose live bindings. The dependency graph is statically analyzable, enabling tooling such as tree shaking.

473. How does es modules work?

Answer

ES modules use static import/export syntax, have module scope, and expose live bindings. The dependency graph is statically analyzable, enabling tooling such as tree shaking. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

474. What are common interview traps with es modules?

Answer

ES modules use static import/export syntax, have module scope, and expose live bindings. The dependency graph is statically analyzable, enabling tooling such as tree shaking. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

475. What is esm?

Answer

ESM is the standardized JavaScript module system using import and export. It has lexical module scope and supports static analysis plus dynamic import().

476. Why does esm matter in production code?

Answer

ESM is the standardized JavaScript module system using import and export. It has lexical module scope and supports static analysis plus dynamic import(). Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

477. What is commonjs?

Answer

CommonJS is Node's traditional module system using require and module.exports. Its loading model is runtime-oriented rather than based on static import declarations.

478. How does commonjs work?

Answer

CommonJS is Node's traditional module system using require and module.exports. Its loading model is runtime-oriented rather than based on static import declarations. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

479. What is live binding?

Answer

ES module imports are live bindings to exported variables. The importing module sees subsequent changes to the exported binding rather than receiving an independent copied value.

480. Why does live binding matter in production code?

Answer

ES module imports are live bindings to exported variables. The importing module sees subsequent changes to the exported binding rather than receiving an independent copied value. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

481. What are common interview traps with live binding?

Answer

ES module imports are live bindings to exported variables. The importing module sees subsequent changes to the exported binding rather than receiving an independent copied value. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

482. What is tree shaking?

Answer

Tree shaking removes code that the bundler can prove is unused. ESM static structure makes this analysis easier than dynamically constructed module dependencies.

483. How does tree shaking work?

Answer

Tree shaking removes code that the bundler can prove is unused. ESM static structure makes this analysis easier than dynamically constructed module dependencies. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

484. What is dynamic import?

Answer

import() loads a module asynchronously and returns a Promise. Bundlers can use it as a code-splitting boundary.

485. Why does dynamic import matter in production code?

Answer

import() loads a module asynchronously and returns a Promise. Bundlers can use it as a code-splitting boundary. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

486. What is code splitting?

Answer

Code splitting divides an application bundle into separately loaded chunks so users need not download all JavaScript up front.

487. How does code splitting work?

Answer

Code splitting divides an application bundle into separately loaded chunks so users need not download all JavaScript up front. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

488. What are common interview traps with code splitting?

Answer

Code splitting divides an application bundle into separately loaded chunks so users need not download all JavaScript up front. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

489. What is module caching?

Answer

Loaded modules are generally evaluated once per module graph/loader context and then reused. Exact caching behavior depends on the runtime and module system.

490. Why does module caching matter in production code?

Answer

Loaded modules are generally evaluated once per module graph/loader context and then reused. Exact caching behavior depends on the runtime and module system. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

491. What is circular dependency?

Answer

A circular dependency exists when module A depends on B while B directly or indirectly depends on A. ESM uses live bindings and ordered evaluation rules, so accessing a binding too early can trigger initialization errors.

492. How does circular dependency work?

Answer

A circular dependency exists when module A depends on B while B directly or indirectly depends on A. ESM uses live bindings and ordered evaluation rules, so accessing a binding too early can trigger initialization errors. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

493. Why is ESM easier to tree-shake than CommonJS?

Answer

CommonJS is Node's traditional module system using require and module.exports. Its loading model is runtime-oriented rather than based on static import declarations. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

494. What are live bindings?

Answer

ES module imports are live bindings to exported variables. The importing module sees subsequent changes to the exported binding rather than receiving an independent copied value.

495. Are imported values copied?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

496. What is top-level await?

Answer

await pauses the current async function until the awaited value is settled, then resumes through Promise continuation semantics. It does not block the JavaScript thread.

497. What is a circular dependency?

Answer

A circular dependency exists when module A depends on B while B directly or indirectly depends on A. ESM uses live bindings and ordered evaluation rules, so accessing a binding too early can trigger initialization errors.

498. Why are ES modules strict by default?

Answer

ES modules use static import/export syntax, have module scope, and expose live bindings. The dependency graph is statically analyzable, enabling tooling such as tree shaking. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

499. What does type: module mean in Node?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

500. Are modules evaluated once?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

19. Node.js Runtime

25 interview prompts

501. What is node.js?

Answer

Node.js is a JavaScript runtime built around V8 plus host facilities such as libuv, filesystem, networking, streams, and process management. It is not itself a JavaScript engine.

502. How does node.js work?

Answer

Node.js is a JavaScript runtime built around V8 plus host facilities such as libuv, filesystem, networking, streams, and process management. It is not itself a JavaScript engine. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

503. What are common interview traps with node.js?

Answer

Node.js is a JavaScript runtime built around V8 plus host facilities such as libuv, filesystem, networking, streams, and process management. It is not itself a JavaScript engine. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

504. What is libuv?

Answer

libuv is the C library underlying much of Node.js async I/O and event-loop behavior. It also provides a worker pool for some operations.

505. How does libuv work?

Answer

libuv is the C library underlying much of Node.js async I/O and event-loop behavior. It also provides a worker pool for some operations. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

506. What is worker threads?

Answer

Node Worker Threads run JavaScript in separate threads, useful for CPU-heavy computation that would otherwise block the main event-loop thread.

507. Why does worker threads matter in production code?

Answer

Node Worker Threads run JavaScript in separate threads, useful for CPU-heavy computation that would otherwise block the main event-loop thread. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

508. What are common interview traps with worker threads?

Answer

Node Worker Threads run JavaScript in separate threads, useful for CPU-heavy computation that would otherwise block the main event-loop thread. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

509. What is child process?

Answer

A child process is a separate OS process with its own memory space. It provides stronger isolation than worker threads but has higher communication and startup costs.

510. How does child process work?

Answer

A child process is a separate OS process with its own memory space. It provides stronger isolation than worker threads but has higher communication and startup costs. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

511. What is stream?

Answer

A stream processes data incrementally rather than requiring the entire payload in memory at once. Node streams support readable, writable, duplex, and transform patterns.

512. Why does stream matter in production code?

Answer

A stream processes data incrementally rather than requiring the entire payload in memory at once. Node streams support readable, writable, duplex, and transform patterns. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

513. What is backpressure?

Answer

Backpressure is a mechanism for slowing producers when consumers cannot keep up, preventing unbounded buffering and memory growth.

514. How does backpressure work?

Answer

Backpressure is a mechanism for slowing producers when consumers cannot keep up, preventing unbounded buffering and memory growth. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

515. What are common interview traps with backpressure?

Answer

Backpressure is a mechanism for slowing producers when consumers cannot keep up, preventing unbounded buffering and memory growth. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

516. What is buffer?

Answer

A Node Buffer is a byte-oriented object used to handle binary data efficiently outside ordinary Unicode JavaScript strings.

517. Why does buffer matter in production code?

Answer

A Node Buffer is a byte-oriented object used to handle binary data efficiently outside ordinary Unicode JavaScript strings. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

518. Is Node.js single-threaded?

Answer

Node.js is a JavaScript runtime built around V8 plus host facilities such as libuv, filesystem, networking, streams, and process management. It is not itself a JavaScript engine.

519. Can Node.js perform parallel work?

Answer

Node.js is a JavaScript runtime built around V8 plus host facilities such as libuv, filesystem, networking, streams, and process management. It is not itself a JavaScript engine.

520. What uses the Node thread pool?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

521. Difference between worker threads and child processes?

Answer

worker threads and child processes solve related problems but differ in semantics and trade-offs. Node Worker Threads run JavaScript in separate threads, useful for CPU-heavy computation that would otherwise block the main event-loop thread. A child process is a separate OS process with its own memory space. It provides stronger isolation than worker threads but has higher communication and startup costs.

522. What is a stream?

Answer

A stream processes data incrementally rather than requiring the entire payload in memory at once. Node streams support readable, writable, duplex, and transform patterns.

523. What is process.nextTick?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

524. What is setImmediate?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

525. How does libuv fit into Node?

Answer

libuv is the C library underlying much of Node.js async I/O and event-loop behavior. It also provides a worker pool for some operations. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

20. Errors and Resilience

9 interview prompts

526. What is error handling?

Answer

Good error handling distinguishes expected failures from programmer bugs, preserves useful context, avoids swallowing errors, and ensures async errors are observed or propagated.

527. How does error handling work?

Answer

Good error handling distinguishes expected failures from programmer bugs, preserves useful context, avoids swallowing errors, and ensures async errors are observed or propagated. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

528. What are common interview traps with error handling?

Answer

Good error handling distinguishes expected failures from programmer bugs, preserves useful context, avoids swallowing errors, and ensures async errors are observed or propagated. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

529. What is throw?

Answer

throw immediately interrupts current synchronous execution and propagates an exception until a matching catch is found or the host reports an uncaught error.

530. How does throw work?

Answer

throw immediately interrupts current synchronous execution and propagates an exception until a matching catch is found or the host reports an uncaught error. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

531. Why does unhandled rejection matter in production code?

Answer

An unhandled rejection is a rejected Promise for which no rejection handler is attached in time. Browsers and Node report these through host-specific global mechanisms. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

532. What are common interview traps with unhandled rejection?

Answer

An unhandled rejection is a rejected Promise for which no rejection handler is attached in time. Browsers and Node report these through host-specific global mechanisms. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

533. How does race condition work?

Answer

A race condition occurs when correctness depends on timing or interleaving of concurrent operations. In UI code, an older response can overwrite newer state unless requests are canceled, sequenced, or ignored when stale. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

534. What are common interview traps with race condition?

Answer

A race condition occurs when correctness depends on timing or interleaving of concurrent operations. In UI code, an older response can overwrite newer state unless requests are canceled, sequenced, or ignored when stale. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

21. Serialization and Data Transfer

13 interview prompts

535. What is serialization?

Answer

Serialization converts in-memory data into a transferable or storable representation; deserialization reconstructs usable data from that representation.

536. How does serialization work?

Answer

Serialization converts in-memory data into a transferable or storable representation; deserialization reconstructs usable data from that representation. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

537. What are common interview traps with serialization?

Answer

Serialization converts in-memory data into a transferable or storable representation; deserialization reconstructs usable data from that representation. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

538. What is json?

Answer

JSON is a text data format with a limited value model. It cannot represent functions, undefined, symbols, cycles, or many built-in types without custom encoding.

539. Why does json matter in production code?

Answer

JSON is a text data format with a limited value model. It cannot represent functions, undefined, symbols, cycles, or many built-in types without custom encoding. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

540. What is structured clone?

Answer

The structured clone algorithm can clone many JavaScript built-ins, including Maps, Sets, Dates, and circular graphs, while intentionally excluding functions and some host objects.

541. How does structured clone work?

Answer

The structured clone algorithm can clone many JavaScript built-ins, including Maps, Sets, Dates, and circular graphs, while intentionally excluding functions and some host objects. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

542. Why does structuredclone matter in production code?

Answer

structuredClone performs the platform structured clone algorithm for many built-in data types and supports circular references. It does not clone functions or DOM nodes. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

543. What are common interview traps with structuredclone?

Answer

structuredClone performs the platform structured clone algorithm for many built-in data types and supports circular references. It does not clone functions or DOM nodes. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

544. What is transferable?

Answer

A transferable object such as an ArrayBuffer can transfer ownership to another execution context instead of being copied, which can reduce overhead for large binary data.

545. How does transferable work?

Answer

A transferable object such as an ArrayBuffer can transfer ownership to another execution context instead of being copied, which can reduce overhead for large binary data. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

546. What is arraybuffer?

Answer

ArrayBuffer is a fixed-length raw binary data buffer. Typed arrays and DataView provide ways to read and write its bytes.

547. Why does arraybuffer matter in production code?

Answer

ArrayBuffer is a fixed-length raw binary data buffer. Typed arrays and DataView provide ways to read and write its bytes. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

22. Symbols, Iterators and Generators

12 interview prompts

548. What is symbol?

Answer

Symbol creates unique primitive identifiers. Symbols are often used for non-colliding property keys and well-known language hooks such as Symbol.iterator.

549. How does symbol work?

Answer

Symbol creates unique primitive identifiers. Symbols are often used for non-colliding property keys and well-known language hooks such as Symbol.iterator. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

550. What are common interview traps with symbol?

Answer

Symbol creates unique primitive identifiers. Symbols are often used for non-colliding property keys and well-known language hooks such as Symbol.iterator. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

551. What is iterator?

Answer

An iterator is an object with a next() method returning {value, done}. It represents a stepwise sequence of values.

552. Why does iterator matter in production code?

Answer

An iterator is an object with a next() method returning {value, done}. It represents a stepwise sequence of values. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

553. What is iterable?

Answer

An iterable is an object that implements Symbol.iterator and can therefore be consumed by for...of, spread, and other iteration protocols.

554. How does iterable work?

Answer

An iterable is an object that implements Symbol.iterator and can therefore be consumed by for...of, spread, and other iteration protocols. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

555. What is generator?

Answer

A generator function creates an iterator whose execution can pause at yield and resume later with next(). It is useful for lazy sequences and cooperative control flow.

556. Why does generator matter in production code?

Answer

A generator function creates an iterator whose execution can pause at yield and resume later with next(). It is useful for lazy sequences and cooperative control flow. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

557. What are common interview traps with generator?

Answer

A generator function creates an iterator whose execution can pause at yield and resume later with next(). It is useful for lazy sequences and cooperative control flow. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

558. What is async iterator?

Answer

An async iterator exposes Symbol.asyncIterator and produces Promises from next(). for await...of consumes these values sequentially.

559. How does async iterator work?

Answer

An async iterator exposes Symbol.asyncIterator and produces Promises from next(). for await...of consumes these values sequentially. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

23. Proxy and Reflect

7 interview prompts

560. What is proxy?

Answer

Proxy wraps a target object and intercepts operations such as get, set, has, ownKeys, and function calls through traps. It is powerful but can complicate reasoning and optimization.

561. How does proxy work?

Answer

Proxy wraps a target object and intercepts operations such as get, set, has, ownKeys, and function calls through traps. It is powerful but can complicate reasoning and optimization. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

562. What are common interview traps with proxy?

Answer

Proxy wraps a target object and intercepts operations such as get, set, has, ownKeys, and function calls through traps. It is powerful but can complicate reasoning and optimization. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

563. What is reflect?

Answer

Reflect exposes standard internal-operation-like methods such as Reflect.get, Reflect.set, and Reflect.ownKeys. Proxy handlers often use Reflect to preserve normal object semantics.

564. Why does reflect matter in production code?

Answer

Reflect exposes standard internal-operation-like methods such as Reflect.get, Reflect.set, and Reflect.ownKeys. Proxy handlers often use Reflect to preserve normal object semantics. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

565. What is proxy invariant?

Answer

Proxy invariants are constraints that prevent a proxy from reporting states that contradict non-configurable target properties or other required object semantics.

566. How does proxy invariant work?

Answer

Proxy invariants are constraints that prevent a proxy from reporting states that contradict non-configurable target properties or other required object semantics. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

24. Debounce, Throttle and Async Patterns

24 interview prompts

567. What is debounce?

Answer

Debounce delays invoking a function until calls stop for a specified interval. It is common for search input, resize handlers, and autosave.

568. How does debounce work?

Answer

Debounce delays invoking a function until calls stop for a specified interval. It is common for search input, resize handlers, and autosave. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

569. What are common interview traps with debounce?

Answer

Debounce delays invoking a function until calls stop for a specified interval. It is common for search input, resize handlers, and autosave. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

570. What is throttle?

Answer

Throttle limits how often a function can run during a period. It is common for scroll, pointermove, or telemetry handlers.

571. Why does throttle matter in production code?

Answer

Throttle limits how often a function can run during a period. It is common for scroll, pointermove, or telemetry handlers. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

572. What is concurrency limit?

Answer

A concurrency limit caps how many async operations may be in flight at once. It can protect APIs, reduce memory usage, and smooth load.

573. How does concurrency limit work?

Answer

A concurrency limit caps how many async operations may be in flight at once. It can protect APIs, reduce memory usage, and smooth load. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

574. What is semaphore?

Answer

A semaphore controls access to a limited number of concurrent slots. An async task acquires a slot before work and releases it afterward.

575. Why does semaphore matter in production code?

Answer

A semaphore controls access to a limited number of concurrent slots. An async task acquires a slot before work and releases it afterward. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

576. What are common interview traps with semaphore?

Answer

A semaphore controls access to a limited number of concurrent slots. An async task acquires a slot before work and releases it afterward. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

577. What is task queue?

Answer

A task queue holds pending work and controls the order and concurrency with which tasks execute. It is a foundation for schedulers and worker pools.

578. How does task queue work?

Answer

A task queue holds pending work and controls the order and concurrency with which tasks execute. It is a foundation for schedulers and worker pools. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

579. What is memoization?

Answer

Memoization caches a function result keyed by its inputs. It helps only when repeated calls are expensive enough and cache lookup plus memory cost are worthwhile.

580. Why does memoization matter in production code?

Answer

Memoization caches a function result keyed by its inputs. It helps only when repeated calls are expensive enough and cache lookup plus memory cost are worthwhile. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

581. Difference between debounce and throttle?

Answer

debounce and throttle solve related problems but differ in semantics and trade-offs. Debounce delays invoking a function until calls stop for a specified interval. It is common for search input, resize handlers, and autosave. Throttle limits how often a function can run during a period. It is common for scroll, pointermove, or telemetry handlers.

582. When would you use leading-edge debounce?

Answer

Debounce delays invoking a function until calls stop for a specified interval. It is common for search input, resize handlers, and autosave.

583. When would you use trailing-edge debounce?

Answer

Debounce delays invoking a function until calls stop for a specified interval. It is common for search input, resize handlers, and autosave.

584. How do you cancel a debounced function?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

585. How do you limit 100 API calls to 5 concurrent requests?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

586. What is a Promise pool?

Answer

A Promise pool keeps a bounded number of async jobs in flight and starts the next job when a slot becomes free. This gives predictable concurrency without launching every operation at once.

587. What is a semaphore?

Answer

A semaphore controls access to a limited number of concurrent slots. An async task acquires a slot before work and releases it afterward.

588. How would you build a task queue?

Answer

A task queue holds pending work and controls the order and concurrency with which tasks execute. It is a foundation for schedulers and worker pools. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

589. How would you deduplicate identical API requests?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

590. How would you enforce latest-request-wins?

Answer

Latest-request-wins means only the result associated with the newest request may update UI state. Common implementations use AbortController, request sequence IDs, or both. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

25. Immutability and Equality

19 interview prompts

591. What is immutability?

Answer

Immutability means existing values are not changed after creation; updates produce new values. It simplifies reasoning and enables cheap reference-based change detection, at a memory-allocation cost.

592. How does immutability work?

Answer

Immutability means existing values are not changed after creation; updates produce new values. It simplifies reasoning and enables cheap reference-based change detection, at a memory-allocation cost. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

593. What are common interview traps with immutability?

Answer

Immutability means existing values are not changed after creation; updates produce new values. It simplifies reasoning and enables cheap reference-based change detection, at a memory-allocation cost. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

594. What is structural sharing?

Answer

Structural sharing reuses unchanged substructures between immutable versions. It reduces copying compared with deep cloning the whole object graph.

595. Why does structural sharing matter in production code?

Answer

Structural sharing reuses unchanged substructures between immutable versions. It reduces copying compared with deep cloning the whole object graph. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

596. What is referential equality?

Answer

Referential equality compares whether two references point to the same object identity, as used by === for objects.

597. How does referential equality work?

Answer

Referential equality compares whether two references point to the same object identity, as used by === for objects. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

598. What is deep equality?

Answer

Deep equality compares nested structure and values rather than top-level identity. It is potentially expensive and must define semantics for cycles, prototypes, Maps, Sets, and special values.

599. Why does deep equality matter in production code?

Answer

Deep equality compares nested structure and values rather than top-level identity. It is potentially expensive and must define semantics for cycles, prototypes, Maps, Sets, and special values. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

600. What are common interview traps with deep equality?

Answer

Deep equality compares nested structure and values rather than top-level identity. It is potentially expensive and must define semantics for cycles, prototypes, Maps, Sets, and special values. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

601. Why does React care about reference equality?

Answer

Explain the component lifecycle, identity comparisons, scheduling implications, and the trade-off between correctness and optimization. Avoid treating React performance rules as absolute laws.

602. Why can mutation make UI updates difficult to detect?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

603. Is immutability always faster?

Answer

Immutability means existing values are not changed after creation; updates produce new values. It simplifies reasoning and enables cheap reference-based change detection, at a memory-allocation cost.

604. When can immutability become expensive?

Answer

Immutability means existing values are not changed after creation; updates produce new values. It simplifies reasoning and enables cheap reference-based change detection, at a memory-allocation cost.

605. Why is deep equality expensive?

Answer

Deep equality compares nested structure and values rather than top-level identity. It is potentially expensive and must define semantics for cycles, prototypes, Maps, Sets, and special values. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

606. Why can new object props defeat memoization?

Answer

Memoization caches a function result keyed by its inputs. It helps only when repeated calls are expensive enough and cache lookup plus memory cost are worthwhile. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

607. What is copy-on-write?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

608. How would you update a deeply nested object immutably?

Answer

An object is a collection of properties identified by keys. Property values can be data, accessors, or functions, and the object can delegate through a prototype. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

609. Why is spread only a shallow copy?

Answer

A shallow copy creates a new outer container but keeps the same nested object references. Spread syntax and Object.assign are shallow copies. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

26. Classes and Private Fields

6 interview prompts

610. What is private field?

Answer

Class private fields use # syntax and are enforced by the language; unlike underscore naming, they cannot be accessed through ordinary property lookup.

611. How does private field work?

Answer

Class private fields use # syntax and are enforced by the language; unlike underscore naming, they cannot be accessed through ordinary property lookup. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

612. What are common interview traps with private field?

Answer

Class private fields use # syntax and are enforced by the language; unlike underscore naming, they cannot be accessed through ordinary property lookup. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

613. What is composition?

Answer

Composition builds behavior by combining smaller units rather than relying on deep inheritance trees. It tends to keep dependencies more explicit and flexible.

614. Why does composition matter in production code?

Answer

Composition builds behavior by combining smaller units rather than relying on deep inheritance trees. It tends to keep dependencies more explicit and flexible. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

615. What are common interview traps with composition?

Answer

Composition builds behavior by combining smaller units rather than relying on deep inheritance trees. It tends to keep dependencies more explicit and flexible. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

27. Browser Storage

14 interview prompts

616. What is localstorage?

Answer

localStorage stores string key-value pairs persistently for an origin. Access is synchronous, so large or frequent operations can block the main thread.

617. How does localstorage work?

Answer

localStorage stores string key-value pairs persistently for an origin. Access is synchronous, so large or frequent operations can block the main thread. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

618. What are common interview traps with localstorage?

Answer

localStorage stores string key-value pairs persistently for an origin. Access is synchronous, so large or frequent operations can block the main thread. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

619. What is sessionstorage?

Answer

sessionStorage stores string key-value pairs scoped to an origin and browser tab/session. Its lifetime is shorter than localStorage.

620. Why does sessionstorage matter in production code?

Answer

sessionStorage stores string key-value pairs scoped to an origin and browser tab/session. Its lifetime is shorter than localStorage. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

621. What is indexeddb?

Answer

IndexedDB is an asynchronous browser database for larger structured data, indexes, transactions, and offline applications.

622. How does indexeddb work?

Answer

IndexedDB is an asynchronous browser database for larger structured data, indexes, transactions, and offline applications. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

623. What is cookie?

Answer

Cookies are small pieces of state attached to HTTP requests according to domain/path and security attributes. They can be HttpOnly, Secure, and SameSite constrained.

624. Why does cookie matter in production code?

Answer

Cookies are small pieces of state attached to HTTP requests according to domain/path and security attributes. They can be HttpOnly, Secure, and SameSite constrained. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

625. What are common interview traps with cookie?

Answer

Cookies are small pieces of state attached to HTTP requests according to domain/path and security attributes. They can be HttpOnly, Secure, and SameSite constrained. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

626. What is httponly?

Answer

HttpOnly prevents JavaScript from reading a cookie through document.cookie. It helps reduce token theft through client-side script but does not eliminate server-side session risks.

627. How does httponly work?

Answer

HttpOnly prevents JavaScript from reading a cookie through document.cookie. It helps reduce token theft through client-side script but does not eliminate server-side session risks. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

628. What is samesite?

Answer

SameSite controls when browsers attach cookies to cross-site requests. Strict and Lax provide stronger CSRF defenses than None, which generally requires Secure.

629. Why does samesite matter in production code?

Answer

SameSite controls when browsers attach cookies to cross-site requests. Strict and Lax provide stronger CSRF defenses than None, which generally requires Secure. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

28. Web Security

22 interview prompts

630. What is xss?

Answer

Cross-site scripting occurs when attacker-controlled content is executed as script in another user's trusted origin context. Defenses include output encoding, safe DOM APIs, CSP, and avoiding unsafe HTML injection.

631. How does xss work?

Answer

Cross-site scripting occurs when attacker-controlled content is executed as script in another user's trusted origin context. Defenses include output encoding, safe DOM APIs, CSP, and avoiding unsafe HTML injection. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

632. What are common interview traps with xss?

Answer

Cross-site scripting occurs when attacker-controlled content is executed as script in another user's trusted origin context. Defenses include output encoding, safe DOM APIs, CSP, and avoiding unsafe HTML injection. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

633. What is csrf?

Answer

Cross-site request forgery tricks a browser into sending an authenticated request from an attacker-controlled site. Defenses include SameSite cookies, CSRF tokens, and validating request context.

634. Why does csrf matter in production code?

Answer

Cross-site request forgery tricks a browser into sending an authenticated request from an attacker-controlled site. Defenses include SameSite cookies, CSRF tokens, and validating request context. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

635. What is cors?

Answer

CORS is a browser security mechanism governing whether frontend JavaScript may read cross-origin responses. The server advertises allowed origins and methods through response headers.

636. How does cors work?

Answer

CORS is a browser security mechanism governing whether frontend JavaScript may read cross-origin responses. The server advertises allowed origins and methods through response headers. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

637. What is preflight?

Answer

A CORS preflight is an OPTIONS request the browser sends before certain cross-origin requests to ask whether the server permits the intended method and headers.

638. Why does preflight matter in production code?

Answer

A CORS preflight is an OPTIONS request the browser sends before certain cross-origin requests to ask whether the server permits the intended method and headers. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

639. What are common interview traps with preflight?

Answer

A CORS preflight is an OPTIONS request the browser sends before certain cross-origin requests to ask whether the server permits the intended method and headers. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

640. What is csp?

Answer

Content Security Policy restricts where scripts and other resources may load from and can use nonces or hashes to reduce the impact of XSS.

641. How does csp work?

Answer

Content Security Policy restricts where scripts and other resources may load from and can use nonces or hashes to reduce the impact of XSS. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

642. What is reflected XSS?

Answer

Cross-site scripting occurs when attacker-controlled content is executed as script in another user's trusted origin context. Defenses include output encoding, safe DOM APIs, CSP, and avoiding unsafe HTML injection.

643. What is stored XSS?

Answer

Cross-site scripting occurs when attacker-controlled content is executed as script in another user's trusted origin context. Defenses include output encoding, safe DOM APIs, CSP, and avoiding unsafe HTML injection.

644. What is DOM-based XSS?

Answer

The DOM is the browser API representation of a document as nodes and objects. It is not the JavaScript language itself; JavaScript interacts with the DOM through host APIs.

645. What is a CORS preflight?

Answer

A CORS preflight is an OPTIONS request the browser sends before certain cross-origin requests to ask whether the server permits the intended method and headers.

646. What triggers a preflight?

Answer

A CORS preflight is an OPTIONS request the browser sends before certain cross-origin requests to ask whether the server permits the intended method and headers.

647. Does CORS protect the server?

Answer

CORS is a browser security mechanism governing whether frontend JavaScript may read cross-origin responses. The server advertises allowed origins and methods through response headers.

648. How do SameSite cookies mitigate CSRF?

Answer

SameSite controls when browsers attach cookies to cross-site requests. Strict and Lax provide stronger CSRF defenses than None, which generally requires Secure. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

649. Why are HttpOnly cookies inaccessible to JavaScript?

Answer

JavaScript is a high-level, dynamically typed language standardized by ECMAScript. Browsers and runtimes such as Node.js execute it through an engine plus host-provided APIs. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

650. What is a CSP nonce?

Answer

Content Security Policy restricts where scripts and other resources may load from and can use nonces or hashes to reduce the impact of XSS.

651. Why is prototype pollution dangerous?

Answer

Prototype pollution is an attack or bug where attacker-controlled input mutates shared prototype objects, causing unexpected inherited properties or behavior across unrelated objects. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

29. Workers and Browser Concurrency

13 interview prompts

652. What is web worker?

Answer

A Web Worker runs JavaScript in a separate thread-like execution context from the page. It cannot directly manipulate the DOM and communicates via messages.

653. How does web worker work?

Answer

A Web Worker runs JavaScript in a separate thread-like execution context from the page. It cannot directly manipulate the DOM and communicates via messages. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

654. What are common interview traps with web worker?

Answer

A Web Worker runs JavaScript in a separate thread-like execution context from the page. It cannot directly manipulate the DOM and communicates via messages. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

655. What is service worker?

Answer

A Service Worker is an event-driven background worker associated with an origin. It can intercept fetches, manage caches, support offline behavior, and handle push-related workflows.

656. Why does service worker matter in production code?

Answer

A Service Worker is an event-driven background worker associated with an origin. It can intercept fetches, manage caches, support offline behavior, and handle push-related workflows. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

657. What is postmessage?

Answer

postMessage sends structured-clonable data between windows, workers, or other supported contexts. It is asynchronous and can optionally transfer ownership of transferable objects.

658. How does postmessage work?

Answer

postMessage sends structured-clonable data between windows, workers, or other supported contexts. It is asynchronous and can optionally transfer ownership of transferable objects. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

659. What is broadcastchannel?

Answer

BroadcastChannel lets same-origin browsing contexts communicate by publishing messages to a named channel.

660. Why does broadcastchannel matter in production code?

Answer

BroadcastChannel lets same-origin browsing contexts communicate by publishing messages to a named channel. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

661. What are common interview traps with broadcastchannel?

Answer

BroadcastChannel lets same-origin browsing contexts communicate by publishing messages to a named channel. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

662. What is messagechannel?

Answer

MessageChannel creates two linked MessagePort objects for direct message passing between contexts.

663. How does messagechannel work?

Answer

MessageChannel creates two linked MessagePort objects for direct message passing between contexts. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

664. Why does transferable matter in production code?

Answer

A transferable object such as an ArrayBuffer can transfer ownership to another execution context instead of being copied, which can reduce overhead for large binary data. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

30. Browser Observers and APIs

17 interview prompts

665. What is mutationobserver?

Answer

MutationObserver batches DOM mutation notifications and delivers them asynchronously, making it preferable to many old mutation event APIs.

666. How does mutationobserver work?

Answer

MutationObserver batches DOM mutation notifications and delivers them asynchronously, making it preferable to many old mutation event APIs. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

667. What are common interview traps with mutationobserver?

Answer

MutationObserver batches DOM mutation notifications and delivers them asynchronously, making it preferable to many old mutation event APIs. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

668. What is resizeobserver?

Answer

ResizeObserver observes changes to an element's box size and reports them asynchronously, avoiding some manual polling approaches.

669. Why does resizeobserver matter in production code?

Answer

ResizeObserver observes changes to an element's box size and reports them asynchronously, avoiding some manual polling approaches. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

670. What is intersectionobserver?

Answer

IntersectionObserver reports when an element enters or leaves an intersection threshold with a root such as the viewport, useful for lazy loading and visibility tracking.

671. How does intersectionobserver work?

Answer

IntersectionObserver reports when an element enters or leaves an intersection threshold with a root such as the viewport, useful for lazy loading and visibility tracking. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

672. What is performanceobserver?

Answer

PerformanceObserver lets code observe entries from the browser Performance Timeline, such as navigation, resource, paint, and long-task measurements.

673. Why does performanceobserver matter in production code?

Answer

PerformanceObserver lets code observe entries from the browser Performance Timeline, such as navigation, resource, paint, and long-task measurements. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

674. What are common interview traps with performanceobserver?

Answer

PerformanceObserver lets code observe entries from the browser Performance Timeline, such as navigation, resource, paint, and long-task measurements. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

675. What is history api?

Answer

The History API lets scripts manipulate session history with pushState and replaceState without a full page load, while popstate reports navigation through history entries.

676. How does history api work?

Answer

The History API lets scripts manipulate session history with pushState and replaceState without a full page load, while popstate reports navigation through history entries. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

677. What is pushstate?

Answer

history.pushState adds a new history entry and changes the URL without reloading the page. It does not itself perform navigation data fetching.

678. Why does pushstate matter in production code?

Answer

history.pushState adds a new history entry and changes the URL without reloading the page. It does not itself perform navigation data fetching. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

679. What is replacestate?

Answer

history.replaceState modifies the current history entry without adding a new one and without reloading the page.

680. How does replacestate work?

Answer

history.replaceState modifies the current history entry without adding a new one and without reloading the page. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

681. What are common interview traps with replacestate?

Answer

history.replaceState modifies the current history entry without adding a new one and without reloading the page. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

31. Networking and Caching

14 interview prompts

682. What is cdn?

Answer

A CDN caches and serves static or dynamic content from geographically distributed edge locations, reducing latency and origin load for many users.

683. How does cdn work?

Answer

A CDN caches and serves static or dynamic content from geographically distributed edge locations, reducing latency and origin load for many users. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

684. What are common interview traps with cdn?

Answer

A CDN caches and serves static or dynamic content from geographically distributed edge locations, reducing latency and origin load for many users. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

685. What is cache-control?

Answer

Cache-Control response directives tell browsers and shared caches how and for how long a response may be reused.

686. Why does cache-control matter in production code?

Answer

Cache-Control response directives tell browsers and shared caches how and for how long a response may be reused. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

687. What is etag?

Answer

ETag is a response validator. A client can send If-None-Match so the server can reply 304 Not Modified when the representation has not changed.

688. How does etag work?

Answer

ETag is a response validator. A client can send If-None-Match so the server can reply 304 Not Modified when the representation has not changed. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

689. What is stale-while-revalidate?

Answer

stale-while-revalidate allows a cache to serve a stale response quickly while revalidating it in the background, trading freshness for latency.

690. Why does stale-while-revalidate matter in production code?

Answer

stale-while-revalidate allows a cache to serve a stale response quickly while revalidating it in the background, trading freshness for latency. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

691. What are common interview traps with stale-while-revalidate?

Answer

stale-while-revalidate allows a cache to serve a stale response quickly while revalidating it in the background, trading freshness for latency. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

692. What is gzip?

Answer

gzip is a widely supported HTTP content-coding based on DEFLATE. It reduces transfer size but differs from Brotli in compression characteristics and typical web performance.

693. How does gzip work?

Answer

gzip is a widely supported HTTP content-coding based on DEFLATE. It reduces transfer size but differs from Brotli in compression characteristics and typical web performance. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

694. What is brotli?

Answer

Brotli is a compression algorithm commonly used for web content. It often provides better compression for text resources than gzip at the cost of additional compression work.

695. Why does brotli matter in production code?

Answer

Brotli is a compression algorithm commonly used for web content. It often provides better compression for text resources than gzip at the cost of additional compression work. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

32. Web Performance Metrics

23 interview prompts

696. What is fcp?

Answer

First Contentful Paint measures when the browser first renders content from the page such as text, image, or canvas pixels.

697. How does fcp work?

Answer

First Contentful Paint measures when the browser first renders content from the page such as text, image, or canvas pixels. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

698. What are common interview traps with fcp?

Answer

First Contentful Paint measures when the browser first renders content from the page such as text, image, or canvas pixels. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

699. What is lcp?

Answer

Largest Contentful Paint measures when the largest visible content element in the initial viewport is rendered, approximating when the main content becomes visible.

700. Why does lcp matter in production code?

Answer

Largest Contentful Paint measures when the largest visible content element in the initial viewport is rendered, approximating when the main content becomes visible. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

701. What is cls?

Answer

Cumulative Layout Shift measures unexpected visual movement of page content during loading and interaction, with penalties based on shift magnitude and impact.

702. How does cls work?

Answer

Cumulative Layout Shift measures unexpected visual movement of page content during loading and interaction, with penalties based on shift magnitude and impact. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

703. What is inp?

Answer

Interaction to Next Paint measures interaction responsiveness by considering the latency from user interaction to the next rendered update across interactions, emphasizing the worst significant delay.

704. Why does inp matter in production code?

Answer

Interaction to Next Paint measures interaction responsiveness by considering the latency from user interaction to the next rendered update across interactions, emphasizing the worst significant delay. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

705. What are common interview traps with inp?

Answer

Interaction to Next Paint measures interaction responsiveness by considering the latency from user interaction to the next rendered update across interactions, emphasizing the worst significant delay. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

706. What is core web vitals?

Answer

Core Web Vitals are Google performance metrics focused on user experience, currently including LCP, CLS, and INP.

707. How does core web vitals work?

Answer

Core Web Vitals are Google performance metrics focused on user experience, currently including LCP, CLS, and INP. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

708. What are common interview traps with long task?

Answer

A long task is a main-thread task long enough to noticeably delay responsiveness, commonly discussed around the 50 ms threshold for performance tooling. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

709. What is TTI?

Answer

Time to Interactive is a historical performance metric describing when a page becomes reliably responsive to user input. Modern performance work usually emphasizes metrics such as INP rather than treating TTI as a primary Core Web Vital.

710. How is LCP different from FCP?

Answer

First Contentful Paint measures when the browser first renders content from the page such as text, image, or canvas pixels. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

711. What causes CLS?

Answer

Cumulative Layout Shift measures unexpected visual movement of page content during loading and interaction, with penalties based on shift magnitude and impact.

712. What does INP measure?

Answer

Interaction to Next Paint measures interaction responsiveness by considering the latency from user interaction to the next rendered update across interactions, emphasizing the worst significant delay.

713. Why can JavaScript execution hurt INP?

Answer

JavaScript is a high-level, dynamically typed language standardized by ECMAScript. Browsers and runtimes such as Node.js execute it through an engine plus host-provided APIs. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

714. What is a long task?

Answer

A long task is a main-thread task long enough to noticeably delay responsiveness, commonly discussed around the 50 ms threshold for performance tooling.

715. How do you reduce main-thread work?

Answer

reduce folds an array into one accumulated result using a callback. It is powerful but can become hard to read when overused for unrelated transformations. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

716. What is hydration cost?

Answer

Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch.

717. How can code splitting improve startup?

Answer

Code splitting divides an application bundle into separately loaded chunks so users need not download all JavaScript up front. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

718. Why can too many small chunks also hurt performance?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

33. SSR, SSG, ISR and Hydration

19 interview prompts

719. What is ssr?

Answer

Server-side rendering generates HTML on the server for a request, which can improve initial content availability and SEO while adding server work and later hydration cost for interactive frameworks.

720. How does ssr work?

Answer

Server-side rendering generates HTML on the server for a request, which can improve initial content availability and SEO while adding server work and later hydration cost for interactive frameworks. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

721. What are common interview traps with ssr?

Answer

Server-side rendering generates HTML on the server for a request, which can improve initial content availability and SEO while adding server work and later hydration cost for interactive frameworks. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

722. What is csr?

Answer

Client-side rendering sends a shell and JavaScript that builds much of the UI in the browser. It can shift work to the client and often increases dependency on JavaScript for initial content.

723. Why does csr matter in production code?

Answer

Client-side rendering sends a shell and JavaScript that builds much of the UI in the browser. It can shift work to the client and often increases dependency on JavaScript for initial content. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

724. What is ssg?

Answer

Static site generation produces HTML ahead of time, typically at build time, and serves those artifacts without rendering every request.

725. How does ssg work?

Answer

Static site generation produces HTML ahead of time, typically at build time, and serves those artifacts without rendering every request. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

726. What is isr?

Answer

Incremental Static Regeneration updates statically generated content after deployment using controlled regeneration rules instead of rebuilding the entire site for every change.

727. Why does isr matter in production code?

Answer

Incremental Static Regeneration updates statically generated content after deployment using controlled regeneration rules instead of rebuilding the entire site for every change. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

728. What are common interview traps with isr?

Answer

Incremental Static Regeneration updates statically generated content after deployment using controlled regeneration rules instead of rebuilding the entire site for every change. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

729. What is hydration?

Answer

Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch.

730. How does hydration work?

Answer

Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

731. What is streaming ssr?

Answer

Streaming SSR sends HTML progressively as parts become ready, allowing the browser to render useful content before the entire response is complete.

732. Why does streaming ssr matter in production code?

Answer

Streaming SSR sends HTML progressively as parts become ready, allowing the browser to render useful content before the entire response is complete. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

733. Why is hydration required?

Answer

Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

734. What is a hydration mismatch?

Answer

Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch.

735. What is partial hydration?

Answer

Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch.

736. What is progressive hydration?

Answer

Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch.

737. How does the browser go from HTML to interactive UI?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

34. React Server Components and Rendering

10 interview prompts

738. What is rsc?

Answer

React Server Components let a framework render some components on the server without shipping those component implementations to the browser. They are different from SSR, which is about rendering HTML for a request.

739. How does rsc work?

Answer

React Server Components let a framework render some components on the server without shipping those component implementations to the browser. They are different from SSR, which is about rendering HTML for a request. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

740. What are common interview traps with rsc?

Answer

React Server Components let a framework render some components on the server without shipping those component implementations to the browser. They are different from SSR, which is about rendering HTML for a request. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

741. What is server components?

Answer

Server Components execute in a server environment and can access server-side resources. Their rendered payload can be combined with client components for interactivity.

742. Why does server components matter in production code?

Answer

Server Components execute in a server environment and can access server-side resources. Their rendered payload can be combined with client components for interactivity. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

743. What is use client?

Answer

In frameworks that support it, "use client" marks a module boundary whose components require client-side execution and can use client-only APIs such as state and effects.

744. How does use client work?

Answer

In frameworks that support it, "use client" marks a module boundary whose components require client-side execution and can use client-only APIs such as state and effects. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

745. Why does hydration matter in production code?

Answer

Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

746. What are common interview traps with hydration?

Answer

Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

747. How does streaming ssr work?

Answer

Streaming SSR sends HTML progressively as parts become ready, allowing the browser to render useful content before the entire response is complete. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

35. React Internals and Rendering

25 interview prompts

748. What is reconciliation?

Answer

Reconciliation is the framework process of comparing the next UI description with the previous one to determine which host changes are needed. React Fiber is the internal architecture used to schedule and perform this work.

749. How does reconciliation work?

Answer

Reconciliation is the framework process of comparing the next UI description with the previous one to determine which host changes are needed. React Fiber is the internal architecture used to schedule and perform this work. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

750. What are common interview traps with reconciliation?

Answer

Reconciliation is the framework process of comparing the next UI description with the previous one to determine which host changes are needed. React Fiber is the internal architecture used to schedule and perform this work. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

751. What is fiber?

Answer

React Fiber is an internal data structure and architecture that represents units of work for the React tree and enables interruptible, prioritized rendering.

752. Why does fiber matter in production code?

Answer

React Fiber is an internal data structure and architecture that represents units of work for the React tree and enables interruptible, prioritized rendering. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

753. What is render phase?

Answer

In React, the render phase computes what the next UI should be. It should be pure because it may run, be restarted, or be abandoned before commit.

754. How does render phase work?

Answer

In React, the render phase computes what the next UI should be. It should be pure because it may run, be restarted, or be abandoned before commit. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

755. What is commit phase?

Answer

The commit phase applies the selected changes to the host environment and runs effect-related commit work. It is where the rendered result becomes externally visible.

756. Why does commit phase matter in production code?

Answer

The commit phase applies the selected changes to the host environment and runs effect-related commit work. It is where the rendered result becomes externally visible. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

757. What are common interview traps with commit phase?

Answer

The commit phase applies the selected changes to the host environment and runs effect-related commit work. It is where the rendered result becomes externally visible. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

758. What is bailout?

Answer

A bailout means skipping work because React can determine a subtree does not need to be recomputed, often using reference equality or memoization boundaries.

759. How does bailout work?

Answer

A bailout means skipping work because React can determine a subtree does not need to be recomputed, often using reference equality or memoization boundaries. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

760. What is concurrent rendering?

Answer

Concurrent rendering lets React work on rendering with scheduling and interruption so higher-priority updates can take precedence. It is an implementation capability, not parallel JavaScript execution of one component.

761. Why does concurrent rendering matter in production code?

Answer

Concurrent rendering lets React work on rendering with scheduling and interruption so higher-priority updates can take precedence. It is an implementation capability, not parallel JavaScript execution of one component. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

762. What is automatic batching?

Answer

React 18 expanded batching so updates from more asynchronous contexts are grouped into fewer renders under concurrent roots, improving efficiency while preserving React's update semantics.

763. How does automatic batching work?

Answer

React 18 expanded batching so updates from more asynchronous contexts are grouped into fewer renders under concurrent roots, improving efficiency while preserving React's update semantics. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

764. What are common interview traps with automatic batching?

Answer

React 18 expanded batching so updates from more asynchronous contexts are grouped into fewer renders under concurrent roots, improving efficiency while preserving React's update semantics. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

765. What causes a React component to rerender?

Answer

Explain the component lifecycle, identity comparisons, scheduling implications, and the trade-off between correctness and optimization. Avoid treating React performance rules as absolute laws.

766. What is the render phase?

Answer

In React, the render phase computes what the next UI should be. It should be pure because it may run, be restarted, or be abandoned before commit.

767. What is the commit phase?

Answer

The commit phase applies the selected changes to the host environment and runs effect-related commit work. It is where the rendered result becomes externally visible.

768. Why must render stay pure?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

769. What is a bailout?

Answer

A bailout means skipping work because React can determine a subtree does not need to be recomputed, often using reference equality or memoization boundaries.

770. What is interruptible rendering?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

771. What changed with automatic batching?

Answer

React 18 expanded batching so updates from more asynchronous contexts are grouped into fewer renders under concurrent roots, improving efficiency while preserving React's update semantics.

772. How do priority and scheduling affect rendering?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

36. React State, Context and Hooks

40 interview prompts

773. What is context?

Answer

React Context broadcasts a value to descendants that read it. Consumers re-render when the provider value they observe changes by reference according to React's context propagation rules.

774. How does context work?

Answer

React Context broadcasts a value to descendants that read it. Consumers re-render when the provider value they observe changes by reference according to React's context propagation rules. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

775. What are common interview traps with context?

Answer

React Context broadcasts a value to descendants that read it. Consumers re-render when the provider value they observe changes by reference according to React's context propagation rules. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

776. What is stale closure?

Answer

A stale closure is a callback that retains values from an earlier render or invocation, so it reads outdated state or props. Functional updates and correct dependency management can prevent common cases.

777. Why does stale closure matter in production code?

Answer

A stale closure is a callback that retains values from an earlier render or invocation, so it reads outdated state or props. Functional updates and correct dependency management can prevent common cases. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

778. What is state colocation?

Answer

State colocation means keeping state near the components that own and use it rather than putting every value in a global store. It can reduce unnecessary propagation and simplify ownership.

779. How does state colocation work?

Answer

State colocation means keeping state near the components that own and use it rather than putting every value in a global store. It can reduce unnecessary propagation and simplify ownership. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

780. What is derived state?

Answer

Derived state is data that can be computed from other state or props. Avoid storing it separately when a direct calculation is sufficient because duplicated sources of truth can drift.

781. Why does derived state matter in production code?

Answer

Derived state is data that can be computed from other state or props. Avoid storing it separately when a direct calculation is sufficient because duplicated sources of truth can drift. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

782. What are common interview traps with derived state?

Answer

Derived state is data that can be computed from other state or props. Avoid storing it separately when a direct calculation is sufficient because duplicated sources of truth can drift. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

783. What is controlled component?

Answer

A controlled form element gets its value from React state and reports changes through events. It makes the source of truth explicit but can increase render activity for very large forms.

784. How does controlled component work?

Answer

A controlled form element gets its value from React state and reports changes through events. It makes the source of truth explicit but can increase render activity for very large forms. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

785. What is uncontrolled component?

Answer

An uncontrolled form element keeps its current value in the DOM and is accessed through a ref or form APIs. It can reduce state plumbing for simple or large forms.

786. Why does uncontrolled component matter in production code?

Answer

An uncontrolled form element keeps its current value in the DOM and is accessed through a ref or form APIs. It can reduce state plumbing for simple or large forms. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

787. What is ref?

Answer

A React ref stores a mutable value that persists across renders without causing a render when it changes. DOM refs expose imperative handles to underlying nodes or components.

788. How does ref work?

Answer

A React ref stores a mutable value that persists across renders without causing a render when it changes. DOM refs expose imperative handles to underlying nodes or components. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

789. What are common interview traps with ref?

Answer

A React ref stores a mutable value that persists across renders without causing a render when it changes. DOM refs expose imperative handles to underlying nodes or components. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

790. What is react memo?

Answer

React.memo memoizes a component render based on props equality, allowing React to skip rendering when props are unchanged according to the comparison strategy.

791. Why does react memo matter in production code?

Answer

React.memo memoizes a component render based on props equality, allowing React to skip rendering when props are unchanged according to the comparison strategy. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

792. What is usememo?

Answer

useMemo caches the result of a calculation between renders when dependencies are unchanged. It is a performance optimization, not a semantic guarantee for preserving state.

793. How does usememo work?

Answer

useMemo caches the result of a calculation between renders when dependencies are unchanged. It is a performance optimization, not a semantic guarantee for preserving state. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

794. What is usecallback?

Answer

useCallback memoizes a function identity between renders when dependencies are unchanged. It is useful mainly when identity affects memoized children or another dependency.

795. Why does usecallback matter in production code?

Answer

useCallback memoizes a function identity between renders when dependencies are unchanged. It is useful mainly when identity affects memoized children or another dependency. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

796. What are common interview traps with usecallback?

Answer

useCallback memoizes a function identity between renders when dependencies are unchanged. It is useful mainly when identity affects memoized children or another dependency. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

797. What is starttransition?

Answer

startTransition marks state updates as non-urgent. React can keep urgent interactions responsive while rendering the transition work at lower priority.

798. How does starttransition work?

Answer

startTransition marks state updates as non-urgent. React can keep urgent interactions responsive while rendering the transition work at lower priority. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

799. What is usedeferredvalue?

Answer

useDeferredValue lets a component receive a lagging version of a value so urgent updates can render first while expensive dependent work is deferred.

800. Why does usedeferredvalue matter in production code?

Answer

useDeferredValue lets a component receive a lagging version of a value so urgent updates can render first while expensive dependent work is deferred. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

801. What is suspense?

Answer

Suspense lets a framework coordinate loading states around components or resources that are not ready. Its exact capabilities depend on the React framework/runtime integration.

802. How does suspense work?

Answer

Suspense lets a framework coordinate loading states around components or resources that are not ready. Its exact capabilities depend on the React framework/runtime integration. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

803. What are common interview traps with suspense?

Answer

Suspense lets a framework coordinate loading states around components or resources that are not ready. Its exact capabilities depend on the React framework/runtime integration. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

804. What causes context consumers to rerender?

Answer

React Context broadcasts a value to descendants that read it. Consumers re-render when the provider value they observe changes by reference according to React's context propagation rules.

805. How can context cause excessive rerenders?

Answer

React Context broadcasts a value to descendants that read it. Consumers re-render when the provider value they observe changes by reference according to React's context propagation rules. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

806. What is a stale closure?

Answer

A stale closure is a callback that retains values from an earlier render or invocation, so it reads outdated state or props. Functional updates and correct dependency management can prevent common cases.

807. Why do dependency arrays use identity?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

808. What happens when an object is a useEffect dependency?

Answer

An object is a collection of properties identified by keys. Property values can be data, accessors, or functions, and the object can delegate through a prototype.

809. Why is ref.current not reactive?

Answer

A React ref stores a mutable value that persists across renders without causing a render when it changes. DOM refs expose imperative handles to underlying nodes or components. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

810. When does useMemo help?

Answer

useMemo caches the result of a calculation between renders when dependencies are unchanged. It is a performance optimization, not a semantic guarantee for preserving state.

811. When can useCallback be useless?

Answer

useCallback memoizes a function identity between renders when dependencies are unchanged. It is useful mainly when identity affects memoized children or another dependency.

812. What is state ownership?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

37. Architecture and Design

16 interview prompts

813. What is dependency injection?

Answer

Dependency injection means supplying a component or function with the dependencies it needs instead of constructing them internally. It improves testability and decoupling.

814. How does dependency injection work?

Answer

Dependency injection means supplying a component or function with the dependencies it needs instead of constructing them internally. It improves testability and decoupling. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

815. What are common interview traps with dependency injection?

Answer

Dependency injection means supplying a component or function with the dependencies it needs instead of constructing them internally. It improves testability and decoupling. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

816. What is inversion of control?

Answer

Inversion of control means a framework or caller controls when/how a component is invoked rather than the component owning the entire flow.

817. Why does inversion of control matter in production code?

Answer

Inversion of control means a framework or caller controls when/how a component is invoked rather than the component owning the entire flow. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

818. How does composition work?

Answer

Composition builds behavior by combining smaller units rather than relying on deep inheritance trees. It tends to keep dependencies more explicit and flexible. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

819. What is design pattern?

Answer

A design pattern is a reusable solution shape for a recurring design problem. Patterns are tools, not rules; use them when they reduce complexity rather than adding ceremony.

820. Why does design pattern matter in production code?

Answer

A design pattern is a reusable solution shape for a recurring design problem. Patterns are tools, not rules; use them when they reduce complexity rather than adding ceremony. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

821. What are common interview traps with design pattern?

Answer

A design pattern is a reusable solution shape for a recurring design problem. Patterns are tools, not rules; use them when they reduce complexity rather than adding ceremony. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

822. What is observer pattern?

Answer

The Observer pattern lets subscribers register for notifications from a subject. It is useful for reactive/evented systems but can make data flow harder to trace if overused.

823. How does observer pattern work?

Answer

The Observer pattern lets subscribers register for notifications from a subject. It is useful for reactive/evented systems but can make data flow harder to trace if overused. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

824. What is pub/sub?

Answer

Publish/subscribe decouples producers from consumers through named channels or topics. Producers publish events without knowing the subscribers directly.

825. Why does pub/sub matter in production code?

Answer

Publish/subscribe decouples producers from consumers through named channels or topics. Producers publish events without knowing the subscribers directly. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

826. What is state machine?

Answer

A state machine models explicit states and allowed transitions. It is useful when UI or business behavior has mutually exclusive states and many conditional branches.

827. How does state machine work?

Answer

A state machine models explicit states and allowed transitions. It is useful when UI or business behavior has mutually exclusive states and many conditional branches. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

828. What are common interview traps with state machine?

Answer

A state machine models explicit states and allowed transitions. It is useful when UI or business behavior has mutually exclusive states and many conditional branches. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

38. Testing

21 interview prompts

829. What is unit testing?

Answer

Unit tests isolate a small unit of behavior and verify deterministic inputs and outputs. They are fast but can miss integration problems.

830. How does unit testing work?

Answer

Unit tests isolate a small unit of behavior and verify deterministic inputs and outputs. They are fast but can miss integration problems. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

831. What are common interview traps with unit testing?

Answer

Unit tests isolate a small unit of behavior and verify deterministic inputs and outputs. They are fast but can miss integration problems. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

832. What is integration testing?

Answer

Integration tests verify how multiple real components or modules work together, catching interface and wiring problems that isolated unit tests miss.

833. Why does integration testing matter in production code?

Answer

Integration tests verify how multiple real components or modules work together, catching interface and wiring problems that isolated unit tests miss. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

834. What is end-to-end testing?

Answer

End-to-end tests exercise the application through a realistic user flow, often including browser, network, and backend integration. They are valuable but usually slower and more brittle.

835. How does end-to-end testing work?

Answer

End-to-end tests exercise the application through a realistic user flow, often including browser, network, and backend integration. They are valuable but usually slower and more brittle. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

836. What is mock?

Answer

A mock is a test double used with expectations about how it is called. In common practice the word is also used more loosely for any test substitute.

837. Why does mock matter in production code?

Answer

A mock is a test double used with expectations about how it is called. In common practice the word is also used more loosely for any test substitute. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

838. What are common interview traps with mock?

Answer

A mock is a test double used with expectations about how it is called. In common practice the word is also used more loosely for any test substitute. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

839. What is stub?

Answer

A stub provides controlled answers to calls so the test can exercise a path without depending on a real dependency.

840. How does stub work?

Answer

A stub provides controlled answers to calls so the test can exercise a path without depending on a real dependency. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

841. What is spy?

Answer

A spy observes calls to a function, such as arguments and call count, while often still allowing the real implementation to execute.

842. Why does spy matter in production code?

Answer

A spy observes calls to a function, such as arguments and call count, while often still allowing the real implementation to execute. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

843. What is fake?

Answer

A fake is a lightweight working implementation that replaces a real dependency, such as an in-memory repository.

844. How does fake work?

Answer

A fake is a lightweight working implementation that replaces a real dependency, such as an in-memory repository. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

845. What are common interview traps with fake?

Answer

A fake is a lightweight working implementation that replaces a real dependency, such as an in-memory repository. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

846. What is fake timer?

Answer

Fake timers replace clock APIs with deterministic virtual time so tests can advance timers without waiting for real time to pass.

847. Why does fake timer matter in production code?

Answer

Fake timers replace clock APIs with deterministic virtual time so tests can advance timers without waiting for real time to pass. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

848. What is flaky test?

Answer

A flaky test sometimes passes and sometimes fails without a relevant code change, often because of timing, shared state, races, environmental dependence, or unstable assertions.

849. How does flaky test work?

Answer

A flaky test sometimes passes and sometimes fails without a relevant code change, often because of timing, shared state, races, environmental dependence, or unstable assertions. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

39. Bundling and Tooling

8 interview prompts

850. What are common interview traps with tree shaking?

Answer

Tree shaking removes code that the bundler can prove is unused. ESM static structure makes this analysis easier than dynamically constructed module dependencies. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

851. Why does code splitting matter in production code?

Answer

Code splitting divides an application bundle into separately loaded chunks so users need not download all JavaScript up front. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

852. How does dynamic import work?

Answer

import() loads a module asynchronously and returns a Promise. Bundlers can use it as a code-splitting boundary. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

853. What is source map?

Answer

A source map maps generated code back to original source locations so development tools can show readable files and stack traces after bundling or transpilation.

854. Why does source map matter in production code?

Answer

A source map maps generated code back to original source locations so development tools can show readable files and stack traces after bundling or transpilation. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

855. What are common interview traps with source map?

Answer

A source map maps generated code back to original source locations so development tools can show readable files and stack traces after bundling or transpilation. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

856. What is tree shaking bundler?

Answer

A bundler builds a module graph, resolves imports, transforms code, and emits deployable assets. Tree shaking and code splitting are common optimization phases.

857. How does tree shaking bundler work?

Answer

A bundler builds a module graph, resolves imports, transforms code, and emits deployable assets. Tree shaking and code splitting are common optimization phases. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

40. Senior Performance and Memory

8 interview prompts

858. How does memoization work?

Answer

Memoization caches a function result keyed by its inputs. It helps only when repeated calls are expensive enough and cache lookup plus memory cost are worthwhile. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

859. What are common interview traps with memoization?

Answer

Memoization caches a function result keyed by its inputs. It helps only when repeated calls are expensive enough and cache lookup plus memory cost are worthwhile. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

860. How does layout thrashing work?

Answer

Layout thrashing occurs when code repeatedly writes style/layout-affecting changes and then synchronously reads layout information, forcing the browser to recalculate layout repeatedly. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

861. What are common interview traps with deoptimization?

Answer

Deoptimization is falling back from optimized machine code when assumptions used by the optimizer are no longer valid, such as a changed type or object shape. Common trap: memorizing the slogan without explaining the underlying execution model or the relevant edge cases.

862. What is virtualization?

Answer

List virtualization renders only the visible portion of a large collection plus a small buffer, reducing DOM size and rendering cost.

863. How does virtualization work?

Answer

List virtualization renders only the visible portion of a large collection plus a small buffer, reducing DOM size and rendering cost. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

864. What is cache invalidation?

Answer

Cache invalidation is deciding when cached data becomes stale and how to update or discard it. It is often the hardest part of caching because correctness depends on freshness guarantees.

865. Why does cache invalidation matter in production code?

Answer

Cache invalidation is deciding when cached data becomes stale and how to update or discard it. It is often the hardest part of caching because correctness depends on freshness guarantees. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

41. Code Output and Scheduling Drills

16 interview prompts

866. Predict the output: console.log("A"); setTimeout(() => console.log("B"), 0); Promise.resolve().then(() => console.log("C")); console.log("D");

Answer

A, D, C, B. Synchronous code runs first; the Promise reaction is a microtask, and the timer callback is a later task.

867. Predict the output: console.log("A"); Promise.resolve().then(() => { console.log("B"); Promise.resolve().then(() => console.log("C")); }); setTimeout(() => console.log("D"), 0); console.log("E");

Answer

A, E, B, C, D. The first Promise reaction runs at the microtask checkpoint and queues another microtask, which runs before the next task.

868. Predict the output: async function test(){ console.log("A"); await Promise.resolve(); console.log("B"); } console.log("C"); test(); console.log("D");

Answer

C, A, D, B. The async function starts synchronously, then code after await is scheduled as a Promise continuation.

869. Predict the output: for (var i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);

Answer

3, 3, 3. var creates one function-scoped binding shared by the callbacks; by the time timers run, the loop has finished.

870. Predict the output: for (let i = 0; i < 3; i++) setTimeout(() => console.log(i), 0);

Answer

0, 1, 2. Each iteration gets a distinct block-scoped binding that the callback closes over.

871. Predict the output: console.log(a); var a = 10;

Answer

undefined. The var binding exists before execution reaches the assignment and is initialized to undefined.

872. Predict the output: console.log(a); let a = 10;

Answer

ReferenceError. The let binding exists but is in the temporal dead zone until initialization.

873. Predict the output: console.log(typeof null); console.log(typeof undefined); console.log(typeof NaN);

Answer

object, undefined, number. typeof null is a historical language quirk; NaN is a Number value.

874. Predict the output: console.log("5" + 2); console.log("5" - 2);

Answer

52 and 3. + concatenates when a string participates after coercion; - performs numeric coercion.

875. Predict the output: console.log(null == undefined); console.log(null === undefined);

Answer

true and false. Abstract equality treats null and undefined as equal to each other only; strict equality compares types too.

876. Predict the output: const a = {}; const b = a; b.x = 1; console.log(a.x);

Answer

1. Both variables reference the same object, so mutating through one reference is visible through the other.

877. Predict the output: const obj = { value: 10, getValue(){ return this.value; } }; const fn = obj.getValue; console.log(fn());

Answer

In sloppy script code it can resolve to a global this value and often prints undefined; in strict/module code this is undefined and accessing this.value throws. The key issue is that detaching the method loses the original receiver.

878. Predict the output: const obj = { a: 1 }; const obj2 = { ...obj }; console.log(obj === obj2);

Answer

false. Spread creates a new outer object with copied enumerable own properties.

879. Predict the output: console.log([] == false); console.log([] === false);

Answer

true and false. Abstract equality performs coercion; strict equality does not.

880. Predict the output: console.log({} == {});

Answer

false. Each object literal creates a distinct object identity.

881. Predict the output: console.log(0.1 + 0.2 === 0.3);

Answer

false in standard JavaScript number arithmetic because 0.1, 0.2, and 0.3 are represented using binary floating-point approximations.

42. High-Frequency Comparison Questions

85 interview prompts

882. What is the difference between var and let?

Interview answer

var and let solve related problems but differ in semantics and trade-offs. var is function-scoped and its binding is initialized to undefined during environment setup; redeclaration is allowed in the same scope. let is block-scoped. Its binding exists before the declaration is executed but remains in the temporal dead zone until initialization.

883. When would you choose var versus let?

Interview answer

Choose based on the behavior you need rather than popularity. var and let differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

884. What is the difference between let and const?

Interview answer

let and const solve related problems but differ in semantics and trade-offs. let is block-scoped. Its binding exists before the declaration is executed but remains in the temporal dead zone until initialization. const is block-scoped and requires initialization at declaration. The binding itself cannot be reassigned, but referenced objects can still be mutated unless they are separately frozen.

885. When would you choose let versus const?

Interview answer

Choose based on the behavior you need rather than popularity. let and const differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

886. When would you choose == versus ===?

Interview answer

Choose based on the behavior you need rather than popularity. == and === differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

887. What is the difference between === and Object.is()?

Interview answer

=== and Object.is() solve related problems but differ in semantics and trade-offs. The distinction depends on the API semantics and execution model. Object.is uses SameValue semantics: it treats NaN as equal to NaN and distinguishes +0 from -0. For most ordinary object comparisons, it still compares object identity.

888. When would you choose === versus Object.is()?

Interview answer

Choose based on the behavior you need rather than popularity. === and Object.is() differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

889. What is the difference between null and undefined?

Interview answer

null and undefined solve related problems but differ in semantics and trade-offs. null is an intentional absence of an object/value. It is a primitive value, although typeof null historically returns "object". undefined means a value is absent or has not been assigned. It is also the default result of many operations that do not produce a value.

890. When would you choose null versus undefined?

Interview answer

Choose based on the behavior you need rather than popularity. null and undefined differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

891. What is the difference between regular function and arrow function?

Interview answer

regular function and arrow function solve related problems but differ in semantics and trade-offs. The distinction depends on the API semantics and execution model. Arrow functions do not create their own this, arguments, or new.target. They capture these from the surrounding lexical context and cannot be used with new.

892. When would you choose regular function versus arrow function?

Interview answer

Choose based on the behavior you need rather than popularity. regular function and arrow function differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

893. What is the difference between call and apply?

Interview answer

call and apply solve related problems but differ in semantics and trade-offs. Function.prototype.call invokes a function immediately with an explicitly supplied this value and individual arguments. Function.prototype.apply invokes a function immediately with an explicitly supplied this value and arguments provided as an array-like object.

894. When would you choose call versus apply?

Interview answer

Choose based on the behavior you need rather than popularity. call and apply differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

895. What is the difference between call and bind?

Interview answer

call and bind solve related problems but differ in semantics and trade-offs. Function.prototype.call invokes a function immediately with an explicitly supplied this value and individual arguments. Function.prototype.bind creates a new function whose this and optionally leading arguments are fixed. Calling bind does not execute the function.

896. When would you choose call versus bind?

Interview answer

Choose based on the behavior you need rather than popularity. call and bind differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

897. What is the difference between apply and bind?

Interview answer

apply and bind solve related problems but differ in semantics and trade-offs. Function.prototype.apply invokes a function immediately with an explicitly supplied this value and arguments provided as an array-like object. Function.prototype.bind creates a new function whose this and optionally leading arguments are fixed. Calling bind does not execute the function.

898. When would you choose apply versus bind?

Interview answer

Choose based on the behavior you need rather than popularity. apply and bind differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

899. What is the difference between prototype and __proto__?

Interview answer

prototype and __proto__ solve related problems but differ in semantics and trade-offs. A function object typically has a prototype property used as the prototype assigned to instances created with new. Methods defined on a class are also stored on a prototype object. __proto__ is a legacy accessor for an object's [[Prototype]]. Object.getPrototypeOf and Object.setPrototypeOf are the standard APIs for inspecting or changing it.

900. When would you choose prototype versus __proto__?

Interview answer

Choose based on the behavior you need rather than popularity. prototype and __proto__ differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

901. What is the difference between slice and splice?

Interview answer

slice and splice solve related problems but differ in semantics and trade-offs. slice returns a shallow copy of a selected range and does not mutate the original array. splice changes an array in place by removing, inserting, or replacing elements.

902. When would you choose slice versus splice?

Interview answer

Choose based on the behavior you need rather than popularity. slice and splice differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

903. What is the difference between map and forEach?

Interview answer

map and forEach solve related problems but differ in semantics and trade-offs. Array.prototype.map creates a new array containing the callback result for each visited element. It does not mutate the original array unless the callback itself mutates something. Array.prototype.forEach calls a callback for each visited element and returns undefined. It is for side effects rather than producing a transformed array.

904. When would you choose map versus forEach?

Interview answer

Choose based on the behavior you need rather than popularity. map and forEach differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

905. What is the difference between find and filter?

Interview answer

find and filter solve related problems but differ in semantics and trade-offs. find returns the first element satisfying a predicate, or undefined if none does. It stops once a match is found. filter creates a new array containing elements for which the predicate returns a truthy result.

906. When would you choose find versus filter?

Interview answer

Choose based on the behavior you need rather than popularity. find and filter differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

907. What is the difference between Promise.all() and Promise.allSettled()?

Interview answer

Promise.all() and Promise.allSettled() solve related problems but differ in semantics and trade-offs. Promise.all waits for all inputs to fulfill and preserves input order in its result array. It rejects as soon as the combined operation is known to have failed, but it does not cancel the underlying operations. Promise.allSettled waits for every input and returns an array describing each fulfilled or rejected outcome.

908. When would you choose Promise.all() versus Promise.allSettled()?

Interview answer

Choose based on the behavior you need rather than popularity. Promise.all() and Promise.allSettled() differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

909. What is the difference between Promise.race() and Promise.any()?

Interview answer

Promise.race() and Promise.any() solve related problems but differ in semantics and trade-offs. Promise.race settles as soon as the first input Promise settles, whether fulfilled or rejected. It does not cancel the other operations. Promise.any fulfills when the first input fulfills and rejects only when all inputs reject, using AggregateError for the combined failure.

910. When would you choose Promise.race() versus Promise.any()?

Interview answer

Choose based on the behavior you need rather than popularity. Promise.race() and Promise.any() differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

911. What is the difference between Promise and async/await?

Interview answer

Promise and async/await solve related problems but differ in semantics and trade-offs. A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final. await pauses the current async function until the awaited value is settled, then resumes through Promise continuation semantics. It does not block the JavaScript thread.

912. When would you choose Promise versus async/await?

Interview answer

Choose based on the behavior you need rather than popularity. Promise and async/await differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

913. What is the difference between microtask and macrotask?

Interview answer

microtask and macrotask solve related problems but differ in semantics and trade-offs. A microtask is a job processed after the current JavaScript stack clears and before the event loop proceeds to a later task. Promise reactions and queueMicrotask are common sources. In browser interview terminology, macrotask commonly means a task such as a timer callback, user interaction, or network event callback. The browser runs tasks one at a time and drains microtasks between tasks.

914. When would you choose microtask versus macrotask?

Interview answer

Choose based on the behavior you need rather than popularity. microtask and macrotask differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

915. What is the difference between setTimeout and requestAnimationFrame?

Interview answer

setTimeout and requestAnimationFrame solve related problems but differ in semantics and trade-offs. Set stores unique values using SameValueZero equality. It is useful for membership checks and deduplication. requestAnimationFrame schedules a callback before a browser paint opportunity, making it appropriate for visual updates synchronized with display refresh.

916. When would you choose setTimeout versus requestAnimationFrame?

Interview answer

Choose based on the behavior you need rather than popularity. setTimeout and requestAnimationFrame differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

917. What is the difference between debounce and throttle?

Interview answer

debounce and throttle solve related problems but differ in semantics and trade-offs. Debounce delays invoking a function until calls stop for a specified interval. It is common for search input, resize handlers, and autosave. Throttle limits how often a function can run during a period. It is common for scroll, pointermove, or telemetry handlers.

918. When would you choose debounce versus throttle?

Interview answer

Choose based on the behavior you need rather than popularity. debounce and throttle differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

919. What is the difference between Map and Object?

Interview answer

Map and Object solve related problems but differ in semantics and trade-offs. Array.prototype.map creates a new array containing the callback result for each visited element. It does not mutate the original array unless the callback itself mutates something. An object is a collection of properties identified by keys. Property values can be data, accessors, or functions, and the object can delegate through a prototype.

920. When would you choose Map versus Object?

Interview answer

Choose based on the behavior you need rather than popularity. Map and Object differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

921. What is the difference between Map and WeakMap?

Interview answer

Map and WeakMap solve related problems but differ in semantics and trade-offs. Array.prototype.map creates a new array containing the callback result for each visited element. It does not mutate the original array unless the callback itself mutates something. WeakMap maps object or non-registered-symbol keys to values without preventing key garbage collection. It is useful for metadata tied to object lifetime.

922. When would you choose Map versus WeakMap?

Interview answer

Choose based on the behavior you need rather than popularity. Map and WeakMap differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

923. What is the difference between Set and WeakSet?

Interview answer

Set and WeakSet solve related problems but differ in semantics and trade-offs. Set stores unique values using SameValueZero equality. It is useful for membership checks and deduplication. WeakSet stores object or non-registered-symbol identities without preventing garbage collection. It is useful for tracking object membership without owning those objects.

924. When would you choose Set versus WeakSet?

Interview answer

Choose based on the behavior you need rather than popularity. Set and WeakSet differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

925. What is the difference between shallow copy and deep copy?

Interview answer

shallow copy and deep copy solve related problems but differ in semantics and trade-offs. A shallow copy creates a new outer container but keeps the same nested object references. Spread syntax and Object.assign are shallow copies. A deep copy recursively duplicates the relevant object graph so nested mutable objects are independent. No single clone strategy works for every JavaScript value.

926. When would you choose shallow copy versus deep copy?

Interview answer

Choose based on the behavior you need rather than popularity. shallow copy and deep copy differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

927. What is the difference between structuredClone and JSON cloning?

Interview answer

structuredClone and JSON cloning solve related problems but differ in semantics and trade-offs. structuredClone performs the platform structured clone algorithm for many built-in data types and supports circular references. It does not clone functions or DOM nodes. JSON is a text data format with a limited value model. It cannot represent functions, undefined, symbols, cycles, or many built-in types without custom encoding.

928. When would you choose structuredClone versus JSON cloning?

Interview answer

Choose based on the behavior you need rather than popularity. structuredClone and JSON cloning differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

929. What is the difference between localStorage and IndexedDB?

Interview answer

localStorage and IndexedDB solve related problems but differ in semantics and trade-offs. localStorage stores string key-value pairs persistently for an origin. Access is synchronous, so large or frequent operations can block the main thread. IndexedDB is an asynchronous browser database for larger structured data, indexes, transactions, and offline applications.

930. When would you choose localStorage versus IndexedDB?

Interview answer

Choose based on the behavior you need rather than popularity. localStorage and IndexedDB differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

931. What is the difference between cookies and localStorage?

Interview answer

cookies and localStorage solve related problems but differ in semantics and trade-offs. Cookies are small pieces of state attached to HTTP requests according to domain/path and security attributes. They can be HttpOnly, Secure, and SameSite constrained. localStorage stores string key-value pairs persistently for an origin. Access is synchronous, so large or frequent operations can block the main thread.

932. When would you choose cookies versus localStorage?

Interview answer

Choose based on the behavior you need rather than popularity. cookies and localStorage differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

933. What is the difference between Web Worker and Service Worker?

Interview answer

Web Worker and Service Worker solve related problems but differ in semantics and trade-offs. A Web Worker runs JavaScript in a separate thread-like execution context from the page. It cannot directly manipulate the DOM and communicates via messages. A Service Worker is an event-driven background worker associated with an origin. It can intercept fetches, manage caches, support offline behavior, and handle push-related workflows.

934. When would you choose Web Worker versus Service Worker?

Interview answer

Choose based on the behavior you need rather than popularity. Web Worker and Service Worker differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

935. What is the difference between Web Worker and Node Worker Thread?

Interview answer

Web Worker and Node Worker Thread solve related problems but differ in semantics and trade-offs. A Web Worker runs JavaScript in a separate thread-like execution context from the page. It cannot directly manipulate the DOM and communicates via messages. Choose based on the exact behavior, lifecycle, and performance constraints required.

936. When would you choose Web Worker versus Node Worker Thread?

Interview answer

Choose based on the behavior you need rather than popularity. Web Worker and Node Worker Thread differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

937. What is the difference between polling and WebSocket?

Interview answer

polling and WebSocket solve related problems but differ in semantics and trade-offs. Polling repeatedly requests or checks for updated state at a chosen interval. It is simple but can waste traffic or add latency compared with push-based approaches. WebSocket provides a persistent, bidirectional connection between client and server after an HTTP-based handshake, useful for low-latency interactive updates.

938. When would you choose polling versus WebSocket?

Interview answer

Choose based on the behavior you need rather than popularity. polling and WebSocket differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

939. What is the difference between SSR and CSR?

Interview answer

SSR and CSR solve related problems but differ in semantics and trade-offs. Server-side rendering generates HTML on the server for a request, which can improve initial content availability and SEO while adding server work and later hydration cost for interactive frameworks. Client-side rendering sends a shell and JavaScript that builds much of the UI in the browser. It can shift work to the client and often increases dependency on JavaScript for initial content.

940. When would you choose SSR versus CSR?

Interview answer

Choose based on the behavior you need rather than popularity. SSR and CSR differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

941. What is the difference between SSR and SSG?

Interview answer

SSR and SSG solve related problems but differ in semantics and trade-offs. Server-side rendering generates HTML on the server for a request, which can improve initial content availability and SEO while adding server work and later hydration cost for interactive frameworks. Static site generation produces HTML ahead of time, typically at build time, and serves those artifacts without rendering every request.

942. When would you choose SSR versus SSG?

Interview answer

Choose based on the behavior you need rather than popularity. SSR and SSG differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

943. What is the difference between hydration and rendering?

Interview answer

hydration and rendering solve related problems but differ in semantics and trade-offs. Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch. Choose based on the exact behavior, lifecycle, and performance constraints required.

944. When would you choose hydration versus rendering?

Interview answer

Choose based on the behavior you need rather than popularity. hydration and rendering differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

945. What is the difference between ESM and CommonJS?

Interview answer

ESM and CommonJS solve related problems but differ in semantics and trade-offs. ESM is the standardized JavaScript module system using import and export. It has lexical module scope and supports static analysis plus dynamic import(). CommonJS is Node's traditional module system using require and module.exports. Its loading model is runtime-oriented rather than based on static import declarations.

946. When would you choose ESM versus CommonJS?

Interview answer

Choose based on the behavior you need rather than popularity. ESM and CommonJS differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

947. What is the difference between mutable and immutable?

Interview answer

mutable and immutable solve related problems but differ in semantics and trade-offs. The distinction depends on the API semantics and execution model. Choose based on the exact behavior, lifecycle, and performance constraints required.

948. When would you choose mutable versus immutable?

Interview answer

Choose based on the behavior you need rather than popularity. mutable and immutable differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

949. What is the difference between shallow equality and deep equality?

Interview answer

shallow equality and deep equality solve related problems but differ in semantics and trade-offs. The distinction depends on the API semantics and execution model. Deep equality compares nested structure and values rather than top-level identity. It is potentially expensive and must define semantics for cycles, prototypes, Maps, Sets, and special values.

950. When would you choose shallow equality versus deep equality?

Interview answer

Choose based on the behavior you need rather than popularity. shallow equality and deep equality differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

951. What is the difference between layout and paint?

Interview answer

layout and paint solve related problems but differ in semantics and trade-offs. The distinction depends on the API semantics and execution model. Choose based on the exact behavior, lifecycle, and performance constraints required.

952. When would you choose layout versus paint?

Interview answer

Choose based on the behavior you need rather than popularity. layout and paint differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

953. What is the difference between paint and composite?

Interview answer

paint and composite solve related problems but differ in semantics and trade-offs. The distinction depends on the API semantics and execution model. Choose based on the exact behavior, lifecycle, and performance constraints required.

954. When would you choose paint versus composite?

Interview answer

Choose based on the behavior you need rather than popularity. paint and composite differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

955. What is the difference between reflow and repaint?

Interview answer

reflow and repaint solve related problems but differ in semantics and trade-offs. Reflow, more precisely layout, computes geometry and positions. DOM or style changes that affect geometry can trigger layout work. Repaint is redrawing pixels after visual properties change without necessarily requiring new layout geometry.

956. When would you choose reflow versus repaint?

Interview answer

Choose based on the behavior you need rather than popularity. reflow and repaint differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

957. What is the difference between CPU-bound and I/O-bound?

Interview answer

CPU-bound and I/O-bound solve related problems but differ in semantics and trade-offs. CPU-bound work is limited primarily by computation time rather than waiting on external I/O. In browsers or Node, move sufficiently heavy CPU work off the main/event-loop thread when responsiveness matters. I/O-bound work spends much of its time waiting on network, disk, or other external resources. Async I/O lets the thread do other work while waiting.

958. When would you choose CPU-bound versus I/O-bound?

Interview answer

Choose based on the behavior you need rather than popularity. CPU-bound and I/O-bound differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

959. What is the difference between unit test and integration test?

Interview answer

unit test and integration test solve related problems but differ in semantics and trade-offs. A unit test isolates a small piece of logic and checks its behavior with controlled inputs. It trades realistic integration coverage for speed, determinism, and focused failure diagnosis. An integration test verifies multiple modules or components working together across their real interfaces. It catches wiring and contract problems that pure unit tests can miss.

960. When would you choose unit test versus integration test?

Interview answer

Choose based on the behavior you need rather than popularity. unit test and integration test differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

961. What is the difference between integration test and end-to-end test?

Interview answer

integration test and end-to-end test solve related problems but differ in semantics and trade-offs. An integration test verifies multiple modules or components working together across their real interfaces. It catches wiring and contract problems that pure unit tests can miss. An end-to-end test exercises a realistic user flow through the application stack, often including browser and backend behavior. It provides high confidence for critical flows at higher execution cost.

962. When would you choose integration test versus end-to-end test?

Interview answer

Choose based on the behavior you need rather than popularity. integration test and end-to-end test differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

963. What is the difference between mock and stub?

Interview answer

mock and stub solve related problems but differ in semantics and trade-offs. A mock is a test double used with expectations about how it is called. In common practice the word is also used more loosely for any test substitute. A stub provides controlled answers to calls so the test can exercise a path without depending on a real dependency.

964. When would you choose mock versus stub?

Interview answer

Choose based on the behavior you need rather than popularity. mock and stub differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

965. What is the difference between stub and spy?

Interview answer

stub and spy solve related problems but differ in semantics and trade-offs. A stub provides controlled answers to calls so the test can exercise a path without depending on a real dependency. A spy observes calls to a function, such as arguments and call count, while often still allowing the real implementation to execute.

966. When would you choose stub versus spy?

Interview answer

Choose based on the behavior you need rather than popularity. stub and spy differ mainly in semantics, lifecycle, and trade-offs; verify the exact contract before optimizing for micro-performance.

43. Senior JavaScript Why/Trade-off Questions

20 interview prompts

967. Why can a Promise callback run before a timer?

Answer

A callback is a function passed to another function or API so it can be invoked later or during a particular operation. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

968. Why does await not block the browser?

Answer

await pauses the current async function until the awaited value is settled, then resumes through Promise continuation semantics. It does not block the JavaScript thread. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

969. Why does fetch not block the call stack?

Answer

The call stack tracks currently executing function calls. JavaScript execution is run-to-completion on a given thread: one stack frame runs until it returns or throws. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

970. Why can microtasks starve rendering?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

971. Why can closures retain memory?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

972. Why can mutation make React rendering harder to reason about?

Answer

Explain the component lifecycle, identity comparisons, scheduling implications, and the trade-off between correctness and optimization. Avoid treating React performance rules as absolute laws.

973. Why can memoization make performance worse?

Answer

Memoization caches a function result keyed by its inputs. It helps only when repeated calls are expensive enough and cache lookup plus memory cost are worthwhile. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

974. Why can object shape affect engine performance?

Answer

An object is a collection of properties identified by keys. Property values can be data, accessors, or functions, and the object can delegate through a prototype. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

975. Why can deep cloning be expensive?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

976. Why is structured cloning different from JSON cloning?

Answer

JSON is a text data format with a limited value model. It cannot represent functions, undefined, symbols, cycles, or many built-in types without custom encoding. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

977. Why is WeakMap useful for object metadata?

Answer

WeakMap maps object or non-registered-symbol keys to values without preventing key garbage collection. It is useful for metadata tied to object lifetime. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

978. Why are ESM dependencies statically analyzable?

Answer

ESM is the standardized JavaScript module system using import and export. It has lexical module scope and supports static analysis plus dynamic import(). Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

979. Why does tree shaking depend on static module structure?

Answer

Tree shaking removes code that the bundler can prove is unused. ESM static structure makes this analysis easier than dynamically constructed module dependencies. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

980. Why does event delegation scale with dynamic lists?

Answer

Event delegation attaches one listener to a common ancestor and uses the event target path to handle many descendant interactions. It reduces listener count and works well with dynamic content. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

981. Why can synchronous localStorage hurt performance?

Answer

localStorage stores string key-value pairs persistently for an origin. Access is synchronous, so large or frequent operations can block the main thread. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

982. Why are Web Workers useful for CPU-heavy tasks?

Answer

A Web Worker runs JavaScript in a separate thread-like execution context from the page. It cannot directly manipulate the DOM and communicates via messages. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

983. Why can too many code-splitting chunks hurt?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

984. Why is cache invalidation difficult?

Answer

Cache invalidation is deciding when cached data becomes stale and how to update or discard it. It is often the hardest part of caching because correctness depends on freshness guarantees. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

985. Why is hydration a separate performance concern from SSR?

Answer

Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

986. Why should render logic in React remain pure?

Answer

Explain the component lifecycle, identity comparisons, scheduling implications, and the trade-off between correctness and optimization. Avoid treating React performance rules as absolute laws.

44. Senior Async and Concurrency Scenarios

12 interview prompts

987. How would you run 100 API calls with a concurrency limit of 10?

Answer

A concurrency limit caps how many async operations may be in flight at once. It can protect APIs, reduce memory usage, and smooth load. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

988. How would you retry only retryable failures?

Answer

Retry repeats a failed operation according to a policy. Production retry logic should bound attempts, distinguish retryable failures, and usually use exponential backoff with jitter. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

989. How would you race a request against a timeout?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

990. How would you prevent duplicate in-flight requests?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

991. How would you cache in-flight Promises?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

992. How would you make latest-request-wins behavior deterministic?

Answer

Latest-request-wins means only the result associated with the newest request may update UI state. Common implementations use AbortController, request sequence IDs, or both. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

993. How would you cancel previous search requests when typing?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

994. How would you handle partial failure in a batch API?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

995. How would you build a Promise pool?

Answer

A Promise pool keeps a bounded number of async jobs in flight and starts the next job when a slot becomes free. This gives predictable concurrency without launching every operation at once. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

996. How would you prevent a retry storm?

Answer

A retry storm happens when many clients retry a failing dependency at once, increasing its load and prolonging the outage. Exponential backoff, jitter, attempt limits, and circuit-breaking patterns reduce the risk. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

997. How would you implement polling with cancellation?

Answer

Polling repeatedly requests or checks for updated state at a chosen interval. It is simple but can waste traffic or add latency compared with push-based approaches. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

998. How would you design a client-side task scheduler?

Answer

The HTML event loop schedules tasks from sources such as timers and user interaction. After a task finishes, the browser performs the required microtask checkpoint before moving on. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

45. Senior Browser Performance Scenarios

10 interview prompts

999. How would you investigate a slow page?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1000. How would you diagnose a memory leak?

Answer

A JavaScript memory leak occurs when objects remain reachable even though the application no longer needs them, so garbage collection cannot reclaim them. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1001. How would you diagnose layout thrashing?

Answer

Layout thrashing occurs when code repeatedly writes style/layout-affecting changes and then synchronously reads layout information, forcing the browser to recalculate layout repeatedly. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1002. How would you reduce long main-thread tasks?

Answer

reduce folds an array into one accumulated result using a callback. It is powerful but can become hard to read when overused for unrelated transformations. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1003. How would you improve a page with a huge JavaScript bundle?

Answer

JavaScript is a high-level, dynamically typed language standardized by ECMAScript. Browsers and runtimes such as Node.js execute it through an engine plus host-provided APIs. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1004. How would you render 100000 rows efficiently?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1005. How would you reduce hydration cost?

Answer

Hydration attaches client-side framework behavior to HTML that was rendered on the server so the page becomes interactive without regenerating every node from scratch. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1006. How would you improve interaction responsiveness?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1007. How would you determine whether a problem is network, CPU, or rendering bound?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1008. How would you use the Performance panel to debug a slow interaction?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

46. Senior React/Frontend Scenarios

12 interview prompts

1009. Why does context sometimes trigger broad rerenders?

Answer

React Context broadcasts a value to descendants that read it. Consumers re-render when the provider value they observe changes by reference according to React's context propagation rules. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

1010. How would you split context to reduce rerenders?

Answer

React Context broadcasts a value to descendants that read it. Consumers re-render when the provider value they observe changes by reference according to React's context propagation rules. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1011. How would you avoid stale closures in event handlers?

Answer

A stale closure is a callback that retains values from an earlier render or invocation, so it reads outdated state or props. Functional updates and correct dependency management can prevent common cases. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1012. How would you choose between local state and a global store?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1013. How would you optimize a large form?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1014. How would you diagnose a component that rerenders too often?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1015. When would you use memoization and when would you remove it?

Answer

Memoization caches a function result keyed by its inputs. It helps only when repeated calls are expensive enough and cache lookup plus memory cost are worthwhile.

1016. How do keys influence reconciliation?

Answer

Reconciliation is the framework process of comparing the next UI description with the previous one to determine which host changes are needed. React Fiber is the internal architecture used to schedule and perform this work. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1017. How would you design a streaming SSR page?

Answer

Streaming SSR sends HTML progressively as parts become ready, allowing the browser to render useful content before the entire response is complete. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1018. How would you separate server and client responsibilities?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1019. How would you avoid unnecessary client JavaScript?

Answer

JavaScript is a high-level, dynamically typed language standardized by ECMAScript. Browsers and runtimes such as Node.js execute it through an engine plus host-provided APIs. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1020. How would you preserve responsive input while expensive derived data updates?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

47. Senior Security Scenarios

8 interview prompts

1021. How would you reduce XSS risk in a React application?

Answer

reduce folds an array into one accumulated result using a callback. It is powerful but can become hard to read when overused for unrelated transformations. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1022. How would you defend a cookie-based session against CSRF?

Answer

Cookies are small pieces of state attached to HTTP requests according to domain/path and security attributes. They can be HttpOnly, Secure, and SameSite constrained. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1023. How would you safely enable a cross-origin frontend?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1024. How would you design a CSP for a frontend app?

Answer

Content Security Policy restricts where scripts and other resources may load from and can use nonces or hashes to reduce the impact of XSS. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1025. Why is storing long-lived secrets in localStorage risky?

Answer

localStorage stores string key-value pairs persistently for an origin. Access is synchronous, so large or frequent operations can block the main thread. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

1026. How can prototype pollution enter a JavaScript application?

Answer

Prototype pollution is an attack or bug where attacker-controlled input mutates shared prototype objects, causing unexpected inherited properties or behavior across unrelated objects. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1027. How would you audit a third-party dependency for supply-chain risk?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1028. How do HttpOnly, Secure, and SameSite work together?

Answer

HttpOnly prevents JavaScript from reading a cookie through document.cookie. It helps reduce token theft through client-side script but does not eliminate server-side session risks. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

48. Senior System Design Vocabulary

18 interview prompts

1029. What is loose coupling?

Answer

Loose coupling means components depend on stable, narrow contracts rather than concrete internal details, making change and testing easier.

1030. What is high cohesion?

Answer

High cohesion means the responsibilities within a module are closely related and reinforce one another. High cohesion with loose coupling generally improves maintainability.

1031. What is separation of concerns?

Answer

Separation of concerns keeps distinct responsibilities in separate modules or layers so each part has a clearer reason to change.

1032. What is an abstraction boundary?

Answer

An abstraction boundary hides implementation details behind a stable interface. Good boundaries reduce the amount of knowledge one component must have about another.

1033. What is encapsulation?

Answer

Encapsulation groups data and behavior behind controlled interfaces so callers do not depend on internal representation.

1034. What is event-driven architecture?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1035. What is a state machine?

Answer

A state machine models explicit states and allowed transitions. It is useful when UI or business behavior has mutually exclusive states and many conditional branches.

1036. What is composition over inheritance?

Answer

Composition builds behavior by combining smaller units rather than relying on deep inheritance trees. It tends to keep dependencies more explicit and flexible.

1037. What is an adapter pattern?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1038. What is a factory pattern?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1039. What is a facade pattern?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1040. What is a middleware pattern?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1041. What is backpressure in a frontend data pipeline?

Answer

Backpressure is a mechanism for slowing producers when consumers cannot keep up, preventing unbounded buffering and memory growth.

1042. What is cache invalidation strategy?

Answer

Cache invalidation is deciding when cached data becomes stale and how to update or discard it. It is often the hardest part of caching because correctness depends on freshness guarantees.

1043. What is request deduplication?

Answer

Request deduplication coalesces identical in-flight requests so multiple callers share one Promise/result instead of creating duplicate network work. The cache key must capture every input that affects the response.

1044. What is idempotence?

Answer

An idempotent operation can be applied multiple times with the same input and have the same externally observable result as applying it once. HTTP method semantics and API retry design often depend on this idea.

1045. What is referential transparency?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1046. What is side effect?

Answer

A side effect is an observable interaction with the outside world beyond computing a returned value, such as mutation, logging, I/O, or DOM changes.

49. Metaprogramming and Language Semantics

22 interview prompts

1047. What is metaprogramming?

Answer

Metaprogramming means writing code that observes or changes how other code behaves or is represented, such as using Proxy, Reflect, symbols, and property descriptors.

1048. What can Proxy intercept?

Answer

Proxy wraps a target object and intercepts operations such as get, set, has, ownKeys, and function calls through traps. It is powerful but can complicate reasoning and optimization.

1049. What can Reflect do?

Answer

Reflect exposes standard internal-operation-like methods such as Reflect.get, Reflect.set, and Reflect.ownKeys. Proxy handlers often use Reflect to preserve normal object semantics.

1050. What are Proxy traps?

Answer

Proxy wraps a target object and intercepts operations such as get, set, has, ownKeys, and function calls through traps. It is powerful but can complicate reasoning and optimization.

1051. What is a Proxy invariant?

Answer

Proxy invariants are constraints that prevent a proxy from reporting states that contradict non-configurable target properties or other required object semantics.

1052. Can Proxy intercept private fields?

Answer

Class private fields use # syntax and are enforced by the language; unlike underscore naming, they cannot be accessed through ordinary property lookup.

1053. What is strict mode?

Answer

Strict mode changes several JavaScript semantics, including disallowing accidental globals and making some invalid operations throw. ES modules are always strict.

1054. What does use strict change?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1055. What is eval()?

Answer

eval executes a string as JavaScript. It harms optimization and readability and creates significant security risk when code is influenced by untrusted input.

1056. Why is eval discouraged?

Answer

eval executes a string as JavaScript. It harms optimization and readability and creates significant security risk when code is influenced by untrusted input. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

1057. What is direct versus indirect eval?

Answer

eval executes a string as JavaScript. It harms optimization and readability and creates significant security risk when code is influenced by untrusted input.

1058. What is new.target?

Answer

new.target lets a constructor or function detect whether it was invoked with new and which constructor was the original target.

1059. What is import.meta?

Answer

import.meta exposes host-provided metadata about the current ES module, such as its module URL in browsers and Node.

1060. What are computed property names?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1061. What are logical assignment operators?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1062. What does ??= do?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1063. What does &&= do?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1064. What does ||= do?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1065. What is an accessor property?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1066. What is a getter and setter?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1067. What is optional chaining evaluation order?

Answer

?. safely accesses a property, call, or index only when the value on its left is not nullish; otherwise the expression evaluates to undefined.

1068. How are property keys coerced?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

50. Date, Number and Internationalization

16 interview prompts

1069. Why is JavaScript Date tricky?

Answer

JavaScript is a high-level, dynamically typed language standardized by ECMAScript. Browsers and runtimes such as Node.js execute it through an engine plus host-provided APIs. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

1070. What is UTC?

Answer

UTC is the time standard used as a global reference. JavaScript Date time values represent instants relative to the UTC-based epoch, while local formatting applies a time zone.

1071. What is a timezone offset?

Answer

A timezone offset is the difference between local civil time and UTC at a given instant. It can change with daylight-saving rules.

1072. What is ISO 8601?

Answer

ISO 8601 is an international date/time representation standard. ISO-like strings such as 2026-09-22T12:34:56Z are commonly used for unambiguous timestamps.

1073. What is Intl?

Answer

Intl is the JavaScript internationalization API namespace for locale-sensitive formatting and related operations.

1074. What is Intl.DateTimeFormat?

Answer

Intl is the JavaScript internationalization API namespace for locale-sensitive formatting and related operations.

1075. What is Intl.NumberFormat?

Answer

Intl is the JavaScript internationalization API namespace for locale-sensitive formatting and related operations.

1076. What is Intl.Collator?

Answer

Intl is the JavaScript internationalization API namespace for locale-sensitive formatting and related operations.

1077. Why should you avoid arbitrary date parsing?

Answer

JavaScript Date represents an instant as a numeric time value. Rendering and parsing still depend on time zones and format rules, so date-only and local-time semantics need explicit handling. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

1078. Why can the same Date display differently across time zones?

Answer

JavaScript Date represents an instant as a numeric time value. Rendering and parsing still depend on time zones and format rules, so date-only and local-time semantics need explicit handling. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

1079. What is epoch time?

Answer

Epoch time is a numeric representation of an instant relative to a defined epoch, commonly milliseconds since 1970-01-01T00:00:00Z in JavaScript Date.

1080. What is a timestamp?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1081. How do you compare dates safely?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1082. Why should date-only values be treated carefully across time zones?

Answer

JavaScript Date represents an instant as a numeric time value. Rendering and parsing still depend on time zones and format rules, so date-only and local-time semantics need explicit handling. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

1083. What is locale-sensitive formatting?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1084. Why should you not hand-roll currency formatting?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

51. Regular Expressions

19 interview prompts

1085. What is a regular expression?

Answer

A regular expression describes text-matching rules. In JavaScript it is implemented by RegExp and can use groups, quantifiers, assertions, and flags.

1086. What is greedy matching?

Answer

A greedy quantifier consumes as much input as it can while still allowing the overall pattern to match.

1087. What is lazy matching?

Answer

A lazy quantifier consumes as little input as possible while still allowing the overall pattern to match.

1088. What are capturing groups?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1089. What are non-capturing groups?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1090. What are lookaheads?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1091. What are lookbehinds?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1092. What is backtracking?

Answer

Backtracking is the regex engine trying alternate paths when a prior choice prevents the rest of the pattern from matching. Poorly designed patterns can cause very large search spaces.

1093. What is catastrophic backtracking?

Answer

Backtracking is the regex engine trying alternate paths when a prior choice prevents the rest of the pattern from matching. Poorly designed patterns can cause very large search spaces.

1094. What is ReDoS?

Answer

ReDoS is Regular Expression Denial of Service: attacker-controlled input triggers excessive regex backtracking and consumes CPU.

1095. How can regex become a denial-of-service risk?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1096. How would you debug a slow regex?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1097. When is regex the wrong tool?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1098. What are regex flags?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1099. What does the g flag do?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1100. What does the m flag do?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1101. What does the s flag do?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1102. What does the u flag do?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1103. What does the y flag do?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

52. Iteration Patterns

15 interview prompts

1104. What is the difference between for, for...in and for...of?

Answer

for, for...in and for...of solve related problems but differ in semantics and trade-offs. The distinction depends on the API semantics and execution model. Choose based on the exact behavior, lifecycle, and performance constraints required.

1105. Why is for...in usually not preferred for arrays?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1106. What does for...of require?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1107. What is the iterator protocol?

Answer

The iterator protocol requires a next() method that returns an object with value and done. Iterables expose an iterator through Symbol.iterator.

1108. What is the iterable protocol?

Answer

The iterable protocol requires Symbol.iterator to return an iterator. It is what for...of and spread use to consume sequences.

1109. What does Symbol.iterator return?

Answer

An iterator is an object with a next() method returning {value, done}. It represents a stepwise sequence of values.

1110. Can you create a custom iterable?

Answer

An iterable is an object that implements Symbol.iterator and can therefore be consumed by for...of, spread, and other iteration protocols.

1111. Can generators implement iterable behavior?

Answer

An iterable is an object that implements Symbol.iterator and can therefore be consumed by for...of, spread, and other iteration protocols.

1112. What is lazy evaluation?

Answer

Lazy evaluation defers producing values until they are requested. Generators and iterators can model lazy sequences without allocating every result up front.

1113. What happens when you spread an iterable?

Answer

An iterable is an object that implements Symbol.iterator and can therefore be consumed by for...of, spread, and other iteration protocols.

1114. Why does Map preserve insertion order?

Answer

Array.prototype.map creates a new array containing the callback result for each visited element. It does not mutate the original array unless the callback itself mutates something. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

1115. How does iteration treat sparse arrays?

Answer

A sparse array has missing indexes (holes). A hole is not the same as an explicit undefined value, and different array methods treat holes differently. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1116. Can you iterate over a Set?

Answer

Set stores unique values using SameValueZero equality. It is useful for membership checks and deduplication.

1117. Can you iterate over a WeakMap?

Answer

WeakMap maps object or non-registered-symbol keys to values without preventing key garbage collection. It is useful for metadata tied to object lifetime.

1118. Why are WeakMap and WeakSet not generally iterable?

Answer

An iterable is an object that implements Symbol.iterator and can therefore be consumed by for...of, spread, and other iteration protocols. Senior angle: explain why it exists, when it matters, and one trade-off or failure mode.

53. Advanced Promise and Concurrency Questions

16 interview prompts

1119. How would you retry only failed requests?

Answer

Retry repeats a failed operation according to a policy. Production retry logic should bound attempts, distinguish retryable failures, and usually use exponential backoff with jitter. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1120. How would you cancel a Promise-backed operation?

Answer

A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1121. How would you prevent stale responses from updating state?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1122. What happens if multiple callers await the same Promise?

Answer

A Promise is an object representing the eventual result of an asynchronous operation. It is pending, then becomes fulfilled or rejected, and settlement is final.

1123. How would you implement exponential backoff with jitter?

Answer

Exponential backoff increases the delay between retries, reducing load on a failing dependency. Jitter randomizes those delays to avoid synchronized retry spikes. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1124. How would you handle partial failures?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1125. How would you make a batch operation resilient?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1126. How would you build a semaphore?

Answer

A semaphore controls access to a limited number of concurrent slots. An async task acquires a slot before work and releases it afterward. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1127. How would you prevent the thundering herd problem?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1128. What is cooperative cancellation?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1129. What is a timeout race?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1130. How would you implement latest-request-wins?

Answer

Latest-request-wins means only the result associated with the newest request may update UI state. Common implementations use AbortController, request sequence IDs, or both. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1131. How would you limit upload concurrency?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1132. How would you sequence dependent API calls?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1133. How would you parallelize independent API calls safely?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1134. How would you avoid accidental sequential await?

Answer

Sequential await waits for one independent operation before starting the next. For independent work, start the Promises first and await Promise.all or a bounded pool to reduce total latency. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

54. Modern JavaScript Architecture

9 interview prompts

1135. What is single responsibility?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1136. What is a singleton pattern?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1137. What is a strategy pattern?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1138. What is an observer pattern?

Answer

The Observer pattern lets subscribers register for notifications from a subject. It is useful for reactive/evented systems but can make data flow harder to trace if overused.

1139. What is a command pattern?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1140. What is a repository pattern?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1141. When should business logic be separated from UI logic?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1142. When does an abstraction become over-engineering?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1143. What is inversion of control in React?

Answer

Inversion of control means a framework or caller controls when/how a component is invoked rather than the component owning the entire flow.

55. Web Storage, Offline and PWA

17 interview prompts

1144. What is a Service Worker?

Answer

A Service Worker is an event-driven background worker associated with an origin. It can intercept fetches, manage caches, support offline behavior, and handle push-related workflows.

1145. How does a Service Worker lifecycle work?

Answer

A Service Worker is an event-driven background worker associated with an origin. It can intercept fetches, manage caches, support offline behavior, and handle push-related workflows. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1146. What are install and activate events?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1147. What happens in the fetch event?

Answer

fetch returns a Promise that fulfills with a Response when the request receives a response, even for HTTP error statuses such as 404. It rejects for failures such as a network error or an abort.

1148. What is CacheStorage?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1149. What is offline-first?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1150. What is stale-while-revalidate caching?

Answer

stale-while-revalidate allows a cache to serve a stale response quickly while revalidating it in the background, trading freshness for latency.

1151. What is background sync?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1152. What is push notification?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1153. What is a PWA?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1154. How do Service Workers intercept requests?

Answer

A Service Worker is an event-driven background worker associated with an origin. It can intercept fetches, manage caches, support offline behavior, and handle push-related workflows. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1155. What happens when a new Service Worker is deployed?

Answer

A Service Worker is an event-driven background worker associated with an origin. It can intercept fetches, manage caches, support offline behavior, and handle push-related workflows.

1156. How do you version Service Worker caches?

Answer

A Service Worker is an event-driven background worker associated with an origin. It can intercept fetches, manage caches, support offline behavior, and handle push-related workflows. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1157. How do you invalidate old caches?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1158. What is IndexedDB used for in offline apps?

Answer

IndexedDB is an asynchronous browser database for larger structured data, indexes, transactions, and offline applications.

1159. What are common offline synchronization challenges?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1160. What is eventual consistency in an offline client?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

56. Debugging and DevTools

20 interview prompts

1161. How do you debug JavaScript?

Answer

JavaScript is a high-level, dynamically typed language standardized by ECMAScript. Browsers and runtimes such as Node.js execute it through an engine plus host-provided APIs. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1162. What is a breakpoint?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1163. What is a conditional breakpoint?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1164. What is a watch expression?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1165. What does the DevTools Call Stack show?

Answer

The call stack tracks currently executing function calls. JavaScript execution is run-to-completion on a given thread: one stack frame runs until it returns or throws.

1166. What does the Scope panel show?

Answer

Scope determines where an identifier can be resolved. JavaScript has global, module, function, and block scopes, with lexical scope based on source-code nesting.

1167. What is a heap snapshot?

Answer

A heap snapshot records the heap graph so you can inspect retained objects and reference paths. Comparing snapshots over time is a common way to diagnose leaks.

1168. How do you find a memory leak?

Answer

A JavaScript memory leak occurs when objects remain reachable even though the application no longer needs them, so garbage collection cannot reclaim them. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1169. How do you find excessive CPU usage?

Answer

find returns the first element satisfying a predicate, or undefined if none does. It stops once a match is found. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1170. How do you find long-running tasks?

Answer

find returns the first element satisfying a predicate, or undefined if none does. It stops once a match is found. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1171. How do you inspect async call stacks?

Answer

The call stack tracks currently executing function calls. JavaScript execution is run-to-completion on a given thread: one stack frame runs until it returns or throws. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1172. How do you debug event listeners?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1173. How do you identify layout thrashing?

Answer

Layout thrashing occurs when code repeatedly writes style/layout-affecting changes and then synchronously reads layout information, forcing the browser to recalculate layout repeatedly. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1174. How do you profile a slow page?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1175. What is a flame chart?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1176. What does a flame chart tell you?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1177. How would you diagnose a slow interaction?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1178. How would you separate network time from CPU time?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1179. How would you debug a race condition?

Answer

A race condition occurs when correctness depends on timing or interleaving of concurrent operations. In UI code, an older response can overwrite newer state unless requests are canceled, sequenced, or ignored when stale. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

1180. How would you use PerformanceObserver in production diagnostics?

Answer

PerformanceObserver lets code observe entries from the browser Performance Timeline, such as navigation, resource, paint, and long-task measurements. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

57. Frontend Networking and Caching Scenarios

20 interview prompts

1181. What is HTTP keep-alive?

Answer

HTTP keep-alive allows a connection to be reused for multiple requests instead of opening a new connection each time, reducing handshake overhead.

1182. What is HTTP/2 multiplexing?

Answer

HTTP/2 multiplexes multiple streams over one connection and adds features such as header compression and stream prioritization semantics.

1183. What is HTTP/3 and QUIC?

Answer

HTTP/3 uses QUIC over UDP and provides transport features designed to reduce some head-of-line blocking and connection setup costs compared with traditional TCP/TLS stacks.

1184. What is DNS?

Answer

DNS maps domain names to network addresses and other records. DNS lookup is one of the stages that can contribute to request latency.

1185. What is TLS?

Answer

TLS provides encrypted, authenticated transport between endpoints, helping provide confidentiality and integrity. HTTPS is HTTP carried over TLS.

1186. What is HTTPS?

Answer

HTTPS is HTTP over TLS. It protects data in transit and authenticates the server through certificates, but it does not make an application secure by itself.

1187. What is connection reuse?

Answer

Connection reuse keeps an established connection available for multiple requests, reducing setup latency and resource consumption.

1188. What is request multiplexing?

Answer

Request multiplexing allows multiple logical requests or streams to share a connection, reducing the need for separate transport connections.

1189. What is compression?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1190. What is CDN caching?

Answer

A CDN caches and serves static or dynamic content from geographically distributed edge locations, reducing latency and origin load for many users.

1191. What is browser memory cache?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1192. What is browser disk cache?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1193. What is cache busting?

Answer

Cache busting changes a resource URL when its content changes, commonly by adding a content hash to the filename. It lets long-lived caches safely reuse old assets while new deployments get distinct URLs.

1194. What is ETag revalidation?

Answer

ETag revalidation uses the entity tag returned by a prior response. The client sends If-None-Match on a later request; if the representation is unchanged, the server can return 304 Not Modified without resending the full body.

1195. What is preloading?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1196. What is prefetching?

Answer

Give the definition, then explain how it behaves at runtime, why it exists, one production example, and one trade-off. That is usually the level expected in a senior JavaScript interview.

1197. What is the difference between preload and prefetch?

Answer

preload and prefetch solve related problems but differ in semantics and trade-offs. Resource preload tells the browser that a resource is important to the current navigation and should be fetched early. It should be used selectively to avoid competing with more important resources. Prefetch hints that a resource may be needed for a future navigation or interaction. It is lower priority than preload and depends on browser/network conditions.

1198. How would you investigate a slow API-backed page?

Answer

Investigate the Network panel and Performance panel together: check DNS/connect/TLS, TTFB, download size, server time, API waterfalls, JavaScript execution, rendering, and long tasks. Fix the dominant bottleneck rather than optimizing a random layer.

1199. How can caching create stale-data bugs?

Answer

Caching creates stale-data bugs when a client reuses a response after the underlying data has changed. Define freshness rules, invalidation triggers, cache keys, and revalidation behavior explicitly.

1200. How would you design cache keys?

Answer

A cache key identifies which request dimensions determine whether cached data can be reused, such as URL, query parameters, headers, or application-specific identity. Senior angle: explain the lifecycle or ordering, the relevant runtime boundary, and the main trade-off.

End of handbook

Total interview questions: 1200.